Algorithmen – exakt beschreiben

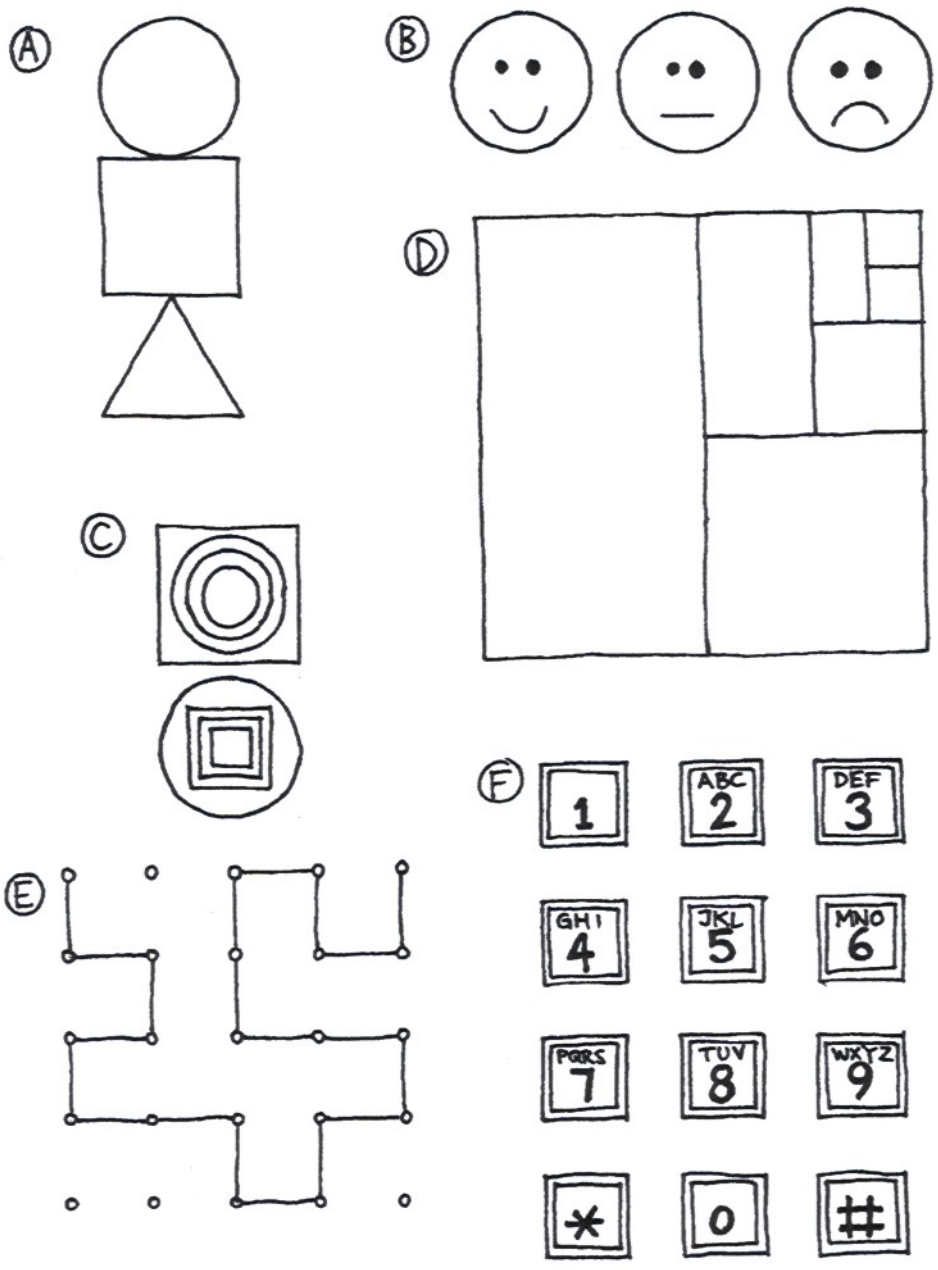
Auftrag für die Gruppen:

1. *Wählt aus eurer Gruppe eine Person X aus, die den anderen aus der Gruppe Anweisungen zum Zeichnen einfacher Bilder geben wird.*
2. *Person X holt sich bei der Lehrerin ein Bild, das sie den anderen nicht zeigt.*
3. *Dann beschreibt X das Bild den anderen möglichst exakt, während diese auf einem karierten Papier versuchen, das Gesagte in ein Bild umzusetzen.*

   ***Achtung:** Begriffe wie Kreis, Quadrat, Dreieck, Linie, Punkt, etc. sind erlaubt, aber keine Begriffe, die eine Aussage über das fertige Produkt treffen, wie Hase, Gesicht, etc.*
4. *Vergleicht dann das Original mit den gezeichneten Bildern. Beantwortet dazu folgende Fragen:*
   1. *Welche Anweisung war nicht verständlich? Wo gab es Probleme bei der Ausführung?*
   2. *Wo und wie hätten die Anweisungen vereinfacht bzw. verkürzt werden können?*
5. *Wechselt dann die Rollen: die oder der nächste der Gruppe holt sich ein weiteres Bild. Wiederholt die Aufträge 3 und 4 so lange, bis jede bzw. jeder von euch einmal ein Bild beschrieben hat.*

Platz für Anmerkungen (Auftrag 4):

Kopiervorlage Bilder:



Quelle: Aus Computer Science Unplugged (*http://csunplugged.org/*)

Algorithmen – Grundelemente

Ein **Algorithmus** ist eine **exakte Verarbeitungsvorschrift**, die aus einer Folge aus Anweisungen besteht.

Grundelemente von Algorithmen



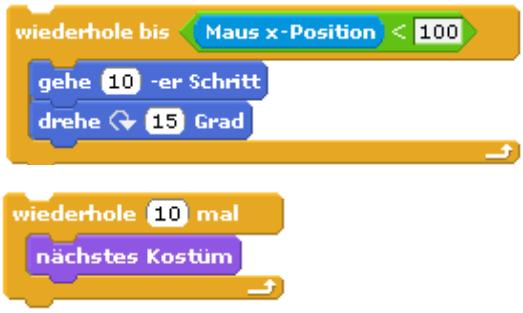
Bezeichnung:

Einsatzmöglichkeiten:



Bezeichnung:

Einsatzmöglichkeiten:



Bezeichnung:

Einsatzmöglichkeiten:

Variablen:

Soll der Computer sich etwas merken, arbeitet man mit Variablen.

1. Erschaffen:



2. Startwert festlegen:



3. Wert ändern:



4. Wert verwenden:



Erste Begegnung mit der Programmiersprache Java

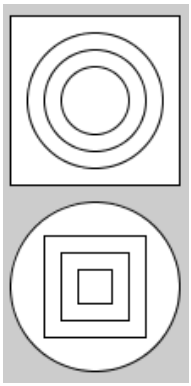
Aufträge:

1. *Öffne die Datei wasmacheich aus dem Tauschverzeichnis in der Entwicklungsumgebung.*
2. ***Stopp! Noch nicht starten!** Lies dir die Zeilen zunächst einmal durch und stelle eine Vermutung auf, was passieren wird, wenn du das Programm startest.*
3. *Starte jetzt das Programm und vergleiche das Ergebnis mit deiner Vermutung. Hattest du Recht?*

4. *Um die folgenden Fragen zu beantworten, kannst du die Zahlen im Programmtext ändern, um zu sehen, was passiert. Merke dir aber immer, wie es vorher ausgesehen hat, damit du notfalls deine Änderungen rückgängig machen kannst.*
5. *Was bewirkt die Anweisung size(b, h) in der setup-Methode?*
6. *Was bewirkt die Anweisung line(x1, y1, x2, y2)?*
7. *Was bewirkt die Anweisung ellipse(x, y, b, h)?*
8. *Was bewirkt die Anweisung rect(x, y, b, h)?*
9. *Die Leinwand, auf der gezeichnet wird, hat ein unsichtbares Koordinatensystem. Wo liegt der Ursprung (0|0)? Gibt es negative Koordinaten? In welche Richtung gehen x- und y-Achse hier? Was ist im Mathematikunterricht anders?*

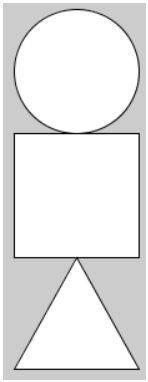
Es ist dir bestimmt aufgefallen, dass jede Anweisung mit einem **Semikolon** (Strichpunkt) abgeschlossen wurde. Wenn du selbst programmierst, darfst du das **nicht vergessen!**

Im Folgenden solltest du dir ein leeres kariertes Blatt Papier bereitlegen, auf dem du zu allem, was du programmieren möchtest, eine kleine Skizze machen kannst. Das erleichtert deine Arbeit sehr!



10. *Jetzt darfst du dein erstes eigenes kleines Programm schreiben. Öffne dazu die Datei figuren aus dem Tauschverzeichnis.*
11. *Füge Programmzeilen so ein, dass das Bild C (Abb. rechts) aus der Gruppenarbeit gezeichnet wird.*
12. *Speichere dein Ergebnis ab.*

13. **** Füge Programmzeilen so ein, dass Bild A (Abb. rechts) gezeichnet wird. Du kannst zum Zeichnen eines Dreiecks die Methode triangle(x1,y1,x2,y2,x3,y3) nutzen. (Es werden die Koordinaten der Eckpunkte des Dreiecks übergeben.)*
14. ****** Überlege dir selbst etwas, was du zeichnen lassen willst und programmiere deine Überlegungen in einer neuen Datei (Datei – Neu). Mache dir vorher eine Skizze auf Papier! Experimentiere auch mit Farbe.*



Nutze für deine Zeichnungen die Informationen zum Zeichnen in deiner Entwicklungsumgebung.

Bilder: Screenshots von Ausführung des Programms „figuren“ (Eisenmann)

Methoden zum Zeichnen

Figuren

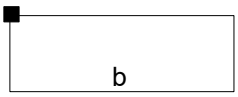
Punkt P(x | y)

```
point(x,y);
```

Strecke PQ mit P(x1 | y1) und Q(x2 | y2)

```
line(x1,y1,x2,y2);
```

Rechteck P(x | y)




```
rect(x,y,b,h);
```

Ellipse mit Mittelpunkt M(x | y), Breite b und Höhe h

```
ellipse(x,y,b,h);
```

Dreieck ABC mit A(x1 | y1), B(x2 | y2) und C(x3 | y3)

```
triangle(x1,y1,x2,y2,x3,y3);
```

Viereck ABCD mit A(x1 | y1), B(x2 | y2) , C(x3 | y3) und D(x4 | y4)

```
quad(x1,y1,x2,y2,x3,y3,x4,y4);
```

Farbe

Achtung:
Zuerst Linien- und Füllfarbe ändern, dann zeichnen lassen!

Füllfarbe in Graustufen:

Schwarz
Weiß
Graustufen

```
fill(0);
fill(255);
fill(a); 0 < a < 255
```

Füllfarbe bunt:

Eingabe einer RGB-Farbe (Werte von Rot, Grün, Blau zwischen 0 und 255)

```
fill(255,0,0); - rot
```

Auswahl/Eingabe einer RGB-Farbe (Tools – Farbauswahl, Farbe wählen und Hexcode kopieren)

```
fill(#FF0000); - rot
```

keine Füllung

```
noFill();
```

Linienfarbe (wie oben)
keine Linie

```
stroke(255); stroke(0,255,0);
noStroke();
```

Hintergrundfarbe einstellen

```
background(122,50,113);
```

Linienstärke

wird mit Pixelanzahl eingegeben

```
strokeWeight(3);
```

Textausgabe

Text an Position (x | y) ausgeben

```
text("Hallo", x, y);
```

Schriftart wählen (Schriftart muss im Setup definiert werden)

```
textFont(kleineSchrift);
```

Zählschleifen in Java

Aus Scratch oder dem MIT App Inventor kennt ihr Kontrollstrukturen, die ein Programm einfacher machen. Natürlich gibt es diese auch in der Programmiersprache Java und wir können sie in unseren Programmen nutzen.

Wird eine Anweisung wiederholt, nutzen wir eine **Schleife**. Wissen wir, wie oft wiederholt werden soll, so bietet sich eine **Zählschleife** an.

In der Programmiersprache Java sieht der Rahmen einer **Zählschleife** folgendermaßen aus:

```
for (int i=0; i<=2; i++) {

}
```

Es gibt dort eine **Zählvariable** i, für die man einen **Startwert**, einen **Endwert** und eine **Schrittweite** angeben kann. Zwischen den geschweiften Klammern steht das, was wiederholt werden soll. Die Zählvariable darf dabei genutzt werden.

In Klammern wird zuerst die Variable i deklariert (bekannt gemacht) und initialisiert (sie bekommt den Wert 0 zugewiesen). Das `int` gibt an, dass i nur ganze Zahlen als Wert annehmen kann.

Dann wird mit i <=2 die Bedingung vorgegeben, dass die Schleife nur durchlaufen wird, wenn i höchstens 2 ist.

Das i++ könnte auch i = i + 1 heißen und bedeutet, dass i nach jedem Schleifendurchlauf um 1 erhöht wird. Also nimmt die Variable i die Werte 0, 1 und 2 in unserem Beispiel an.

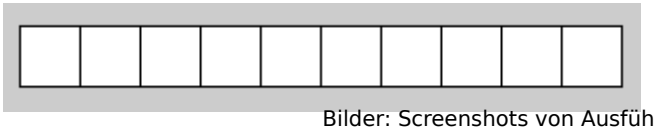
Aufträge:

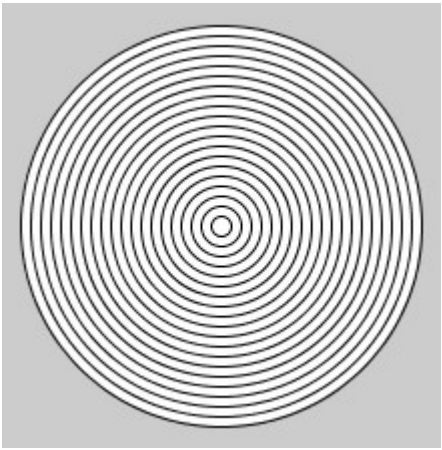
1. *Ergänze in der Vorlage figurC_mitSchleife aus dem Tauschverzeichnis die Methode zeichneBildC() so, dass zunächst die drei Kreise in das Quadrat gezeichnet werden. Nutze dazu eine Zählschleife.*
2. *Für die drei Quadrate in dem anderen Kreis kannst du auch eine Schleife verwenden. Suche wieder nach den Regelmäßigkeiten für den Startpunkt der Quadrate und für die Seitenlänge (bzw. Breite) und schreibe dein Programm um.*
   *(Bei deiner Lehrerin gibt es **Hilfekarten**, wenn du Hilfe brauchst!)*

Weitere Übungen

Nutze für die weiteren Übungen eine neue Datei, in der du die eigenen Methoden schreibst und in der setup()-Methode aufrufst.

3. *Schreibe eine Methode zeichneQuadrate(), die 10 Quadrate gleicher Größe direkt nebeneinander zeichnet.*
4. *Schreibe eine Methode zeichneKreise(), die 20 Kreise ineinander, die immer kleiner werden, zeichnet.*





Bilder: Screenshots von Ausführung des Programms „zehn_quadrate“ (Eisenmann)

**** Schreibe die beiden Methoden aus 3. und 4. so, dass man ihnen die Anzahl der Quadrate bzw. Kreise übergeben kann. Hol dir dazu das Infoblatt zu Methoden in Java.*

HILFEKARTEN ZU AUFGABE 2

Tipp 1

Bei den Quadraten ändert sich sowohl der Startpunkt als auch die Seitenlänge.

Versuche zuerst, x- und y-Koordinate des Eckpunktes zu beschreiben.

Tipp 2

Beim ersten Quadrat ist x = 30 und y = 140.

Beim zweiten Quadrat ist x = 40 und y = 150.

Überlege dir auch die Koordinaten beim dritten Quadrat und suche nach der Veränderung.

Tipp 3

Die Veränderung ist 10 bei beiden Koordinaten.

In der Schleife bei i = 0 bleibt es bei 30 und 140, bei i = 1, wird jeweils 1 * 10 addiert, bei i = 2, wird 2 * 10 addiert, usw.

Versuche, die beiden Koordinaten mit i auszudrücken.

Tipp 4

x berechnet sich durch 30 + i * 10

y berechnet sich durch 140 + i * 10

Überlege dir jetzt, wie du auf die beiden restlichen Werte (die Breite und Höhe der Quadrate) kommst.

Tipp 5

Die beiden Werte sind gleich, da es sich um Quadrate handelt.

Beim ersten Quadrat ist der Wert 60, dann 40, dann 20.

Beschreibe den Wert wieder mithilfe der Schleifenvariable i.

Tipp 6

Breite und Höhe lassen sich durch 60 – i * 20 berechnen.

Lösung

```
rect(30+i*10, 140+i*10, 60-i*20, 60-i*20);
```

Solltest du den Weg nicht verstanden haben, sieh dir das Video „Zählschleife Quadrat“ an.

```
if (klassenstufe == 9) {
   fachwahl="IMP";
}
```

Folie für die Plenumsphase zu Beginn

```
rect(10, 10, 100, 100);

ellipse(60, 60, 80, 80);

ellipse(60, 60, 60, 60);

ellipse(60, 60, 40, 40);



  ellipse(                    );
```

Zählschleifen in Java – weitere Übungen

Zur Erinnerung:

In der Programmiersprache Java sieht der Rahmen einer **Zählschleife** folgendermaßen aus:

```
for (int i=0; i<=2; i++) {

}
```

Es gibt dort eine **Zählvariable** i, für die man einen **Startwert**, einen **Endwert** und eine **Schrittweite** angeben kann. Zwischen den geschweiften Klammern steht das, was wiederholt werden soll. Die Zählvariable darf dabei genutzt werden.

In Klammern wird zuerst die Variable i deklariert (bekannt gemacht) und initialisiert (sie bekommt den Wert 0 zugewiesen). Das `int` gibt an, dass i nur ganze Zahlen als Wert annehmen kann.

Dann wird mit i <=2 die Bedingung vorgegeben, dass die Schleife nur durchlaufen wird, wenn i höchstens 2 ist.

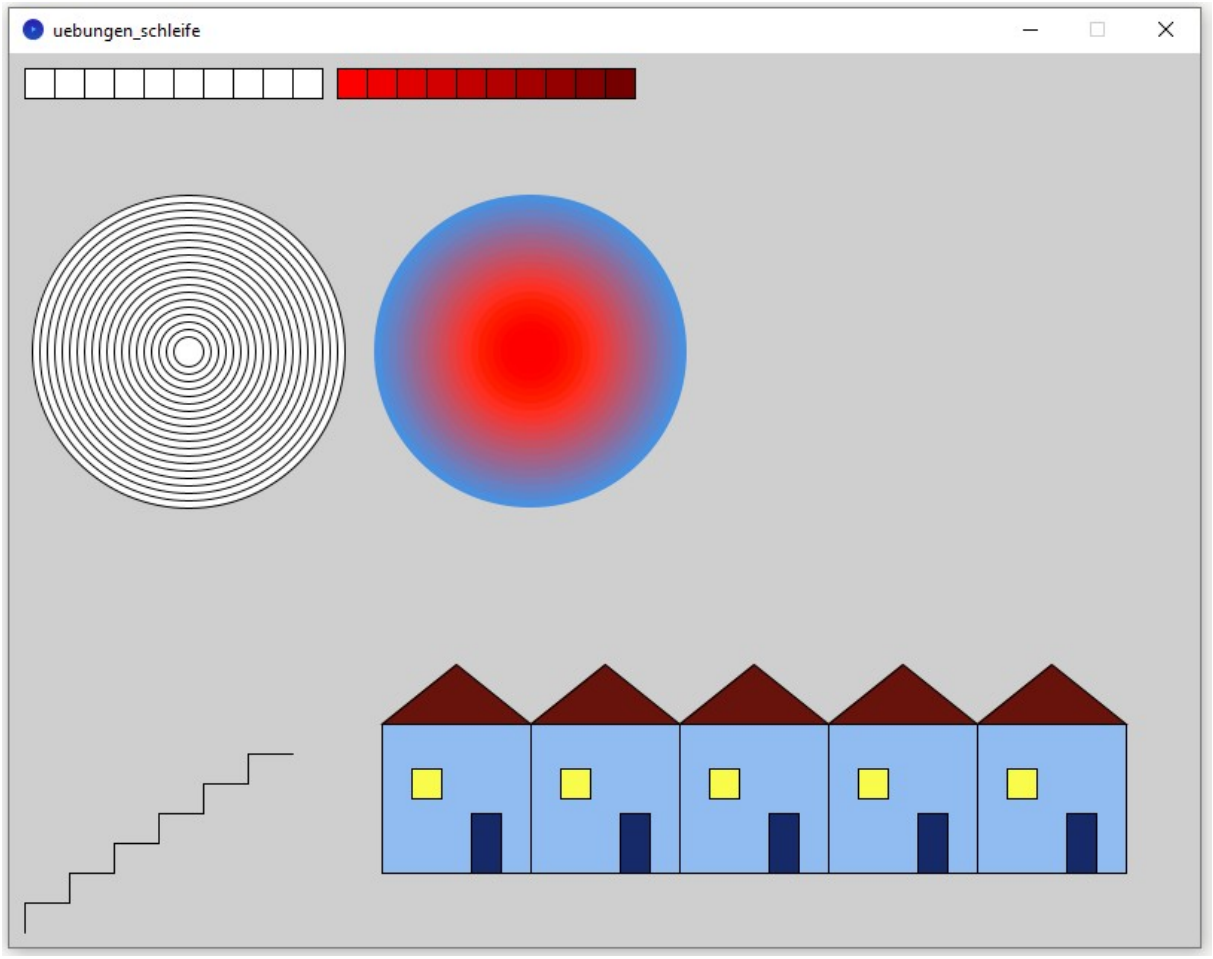
Das i++ könnte auch i = i + 1 heißen und bedeutet, dass i nach jedem Schleifendurchlauf um 1 erhöht wird. Also nimmt die Variable i die Werte 0, 1 und 2 in unserem Beispiel an.

Weitere Übungen

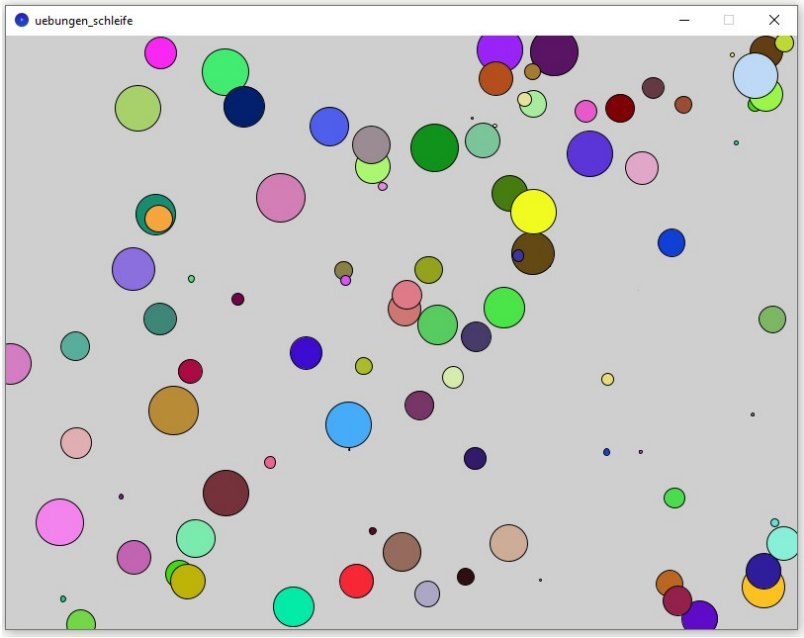
Nutze für die weiteren Übungen die Datei `uebungen_schleife`. Ergänze dazu jeweils die genannte Methode.

1. *Teil a) Schreibe eine Methode zeichneQuadrate(), die 10 Quadrate gleicher Größe direkt nebeneinander zeichnet.*
   Teil b) Schreibe eine Methode zeichneFarbQuadrate(), die 10 weitere Quadrate gleicher Größe nebeneinander zeichnet, die von links nach rechts immer dunkler werden. (Denke daran, dass eine Farbe aus Rot-, Grün- und Blauanteil (0 bis 255) besteht. Du kannst auch hier schrittweise etwas abziehen.)
2. *Teil a) Schreibe eine Methode zeichneKreise(), die 20 Kreise ineinander, die immer kleiner werden, zeichnet. (Denke daran, die Farbe erst wieder zu ändern: fill(255)).*
   Teil b) Schreibe eine Methode zeichneFarbKreise(), die 10 Kreise ineinander zeichnet, die immer dunkler (oder auch heller) werden. (Tipp: Nutze nur Füllfarbe und keine Stiftfarbe (noStroke()). Im Beispiel wurden alle Farbwerte schrittweise verändert. Experimentiere!)
3. *Schreibe eine Methode zeichneTreppenstufen(), die eine bestimmte Anzahl an Treppenstufen zeichnet, die nach oben führen. (Stiftfarbe vorgeben! - z.B. stroke(0) für schwarz)*
4. *Schreibe eine Methode reihenhaussiedlung(), die fünf gleiche Häuser nebeneinander zeichnet.*
5. *Schreibe eine Methode seifenblasen(), die 100 verschiedenfarbige Kreise zufällig auf dem Fenster zeichnet. (Hier nutzt du nicht die Zählvariable zum Berechnen der Position, sondern Zufallszahlen. random(255) erzeugt z.B. eine Zufallszahl zwischen 0 und 255.)*

Wie das Ergebnis der Aufgaben aussehen könnte, siehst du auf der Abbildung auf der Rückseite des Blattes.



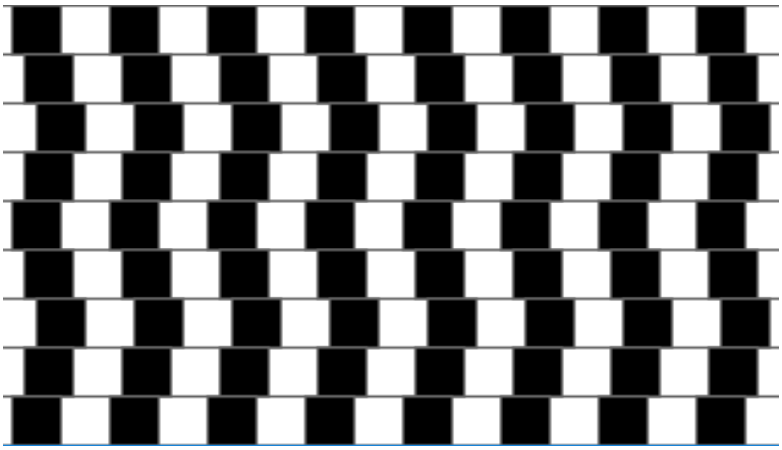
Bilder: Screenshots von Ausführung des Programms „uebungen_schleife“ (Eisenmann)



Optische Täuschungen

Eine optische Täuschung ist eine Wahrnehmungstäuschung des Sehsinns. Sie kann viele Aspekte des Sehens betreffen. Es gibt z.B. Tiefenillusionen, Farbillusionen, geometrische Illusionen und Bewegungsillusionen[1].

Vielleicht habt ihr im Mathematikunterricht schon einmal eine optische Täuschung selbst hergestellt, als ihr in Klasse 5 das Zeichnen von parallelen und orthogonalen Linien üben solltet. Es gibt nämlich solche, die aus parallelen (und senkrechten) Geraden bestehen, aber ganz krumm aussehen, wie z.B. die sogenannte Münsterberg- oder auch Kaffeehaus-Täuschung. Diese werden wir uns genauer anschauen und sie dann selbst programmieren.



Bilder: Screenshots von Ausführung des Programms „kaffeehaus“ (Eisenmann)

1. *Überprüfe zunächst mit deinem Geodreieck, ob es sich tatsächlich um parallele Geraden handelt.*
2. *Speichere ein neues Programm unter „kaffeehaus“ ab.*
3. *Schreibe zunächst den Rahmen der setup-Methode, gib die Größe des Fensters (640x400) an und lasse den Hintergrund weiß „färben“.*
4. *Schreibe eine Methode parallelen(), die zehn graue Parallelen im Abstand von 40 Pixel zeichnet. Nutze dazu eine Zählschleife. Vergleiche dein Ergebnis mit der ausliegenden Lösung.*

Hole dir jetzt Blatt 2. Darauf erfährst du Schritt für Schritt, wie es weitergehen könnte und lernst auch weitere Elemente von Java kennen.

1 Siehe Seite „Optische Täuschung“. In: Wikipedia, Die freie Enzyklopädie. Bearbeitungsstand: 11. September 2018, 16:14 UTC. URL: *https://de.wikipedia.org/w/index.php?title=Optische_T%C3%A4uschung&oldid=180828619* (Abgerufen: 14. September 2018, 15:58 UTC)

Du siehst hier wieder diese grau geschriebenen Hinweise. Diese heißen **Kommentare**. Sie sind dazu da, um dein Programm lesbarer zu gestalten. So kann auch jemand anderes schnell erkennen, was in deinem Programm passiert.

Ein Kommentar wird immer nach zwei Schrägstrichen geschrieben, wenn er in eine Zeile passt. Dein Computer ignoriert beim Übersetzen deines Programms die Kommentare. Bsp.:

```
background(255); // Hintergrund weiß
parallelen();  // Aufruf der Methode
```

Es gibt auch noch eine weitere Möglichkeit, einen Kommentar zu kennzeichnen.

```
/* auch das ist ein Kommentar, der
   sogar über mehrere Zeilen
   gehen kann */
```

Ist etwas zwischen /* und */ eingeschlossen, wird es auch beim Übersetzen ignoriert.

So kannst du auch mal einen Programmteil „auskommentieren“, wenn etwas nicht funktioniert, ohne ihn gleich löschen zu müssen.

5. *Ergänze in deinem Programm sinnvolle Kommentare. Du kannst die aus der ausliegenden Lösung als Beispiel nehmen.*

Den Abstand der Linien haben wir auf 40 festgelegt. Diesen Wert brauchen wir auch später als Seitenlänge der vielen Quadrate, die wir noch zeichnen lassen werden. Sollten wir auf die Idee kommen, die Quadrate kleiner oder größer haben zu wollen, müssten wir an vielen Stellen etwas ändern.

Deshalb nutzen wir eine **Variable**, die im ganzen Programm bekannt ist. Im Gegensatz zu der Zählvariable `i`, die nur in der Schleife (**lokal**) bekannt ist, ist die neue Variable überall (**global**) bekannt. Wir nennen sie `s` für Seitenlänge und geben ihr den Wert 40. Auch diese Variable ist eine ganze Zahl, sie bekommt deshalb auch den Typ `int`.

Die Deklaration globaler Variablen (auch möglich mit Initialisierung) wird an den Anfang des Programms geschrieben.

Es kommt immer zuerst der Datentyp (z.B. int für eine ganze Zahl), dann der Name der Variable, dann ein Strichpunkt oder schon die Initialisierung der Variable.

Beispiele: `int start;` oder: `int start = 1;`

6. *Verändere deinen Programmcode, indem du die Variable s für die Seitenlänge der Quadrate bzw. den Abstand der Parallelen einführst und nutzt. Vergleiche dein Ergebnis mit der ausliegenden Lösung.*

Hol dir jetzt das nächste Blatt.

Blatt 2 – Die Quadrate...

Als Nächstes füllen wir die Reihen mit Quadraten.

7. *Schreibe den Rahmen einer Methode zeichneQuadrate(). Stelle darin zunächst schwarz als Füllfarbe ein. (Die Linienfarbe bleibt grau.)*

In einer Reihe sollen jeweils acht Quadrate gezeichnet werden, die wieder den Abstand der eigenen Seitenlänge haben.

8. *Programmiere das Zeichnen der Quadrate in der ersten Reihe. Nutze wieder eine Zählschleife und nutze auch die Variable s für die Seitenlänge. Das erste Quadrat soll einen Abstand von 10 Pixeln zum Rand haben. Starte dein Programm und ändere deinen Programmcode ab, wenn etwas noch nicht stimmt. (Die Lösung liegt aus.)*

In jeder Reihe startet das erste Quadrat aber an einer anderen Stelle. In der ersten, mittleren und letzten Reihe bei 10 Pixeln, jeweils daneben bei 20 und in der dritten und siebten Reihe bei 30 Pixel.

9. *Überlege dir, was du für die nächsten Reihen ändern musst.*
10. *Programmiere das Zeichnen der Quadrate in der zweiten Reihe.*
11. *Programmiere das Zeichnen der Quadrate in der dritten Reihe.*

Vergleiche mit der ausliegenden Lösung und hol dir dann das nächste Blatt.

Es ist sehr umständlich, für jede Reihe eine neue Schleife programmieren zu müssen. Eigentlich wiederholen wir ja wieder neun Mal das Zeichnen einer Reihe von Quadraten.

12. Gib an, was gleich bleibt und überlege dir, was sich ändert. Fülle dazu folgende Tabelle weiter aus:

Reihe	x	y	b	h
Erste Reihe	10 + 2 * s * i	5	s	s
Zweite Reihe	20 + 2 * s * i	5 + s	s	s

Wir ignorieren zunächst einmal die unterschiedlichen Abstände zum linken Rand und nehmen bei allen Reihen die Zahl 10. Um die Änderung kümmern wir uns später.

Dann ändert sich von Reihe zu Reihe nur die y-Koordinate.

13. Überlege dir, wie du mit einer weiteren Zählvariable j auf die y-Koordinaten kommst. Vergleiche deine Idee mit der ausliegenden Lösung.

Wir bekommen die Quadrate einer Reihe mithilfe einer Zählschleife, die sich in der vereinfachten Form nur in der y-Koordinate unterscheidet.

Da wir aber neun solcher Reihen brauchen, nutzen wir eine weitere Zählschleife, die neunmal unsere bisherige Zählschleife durchläuft.

Sind zwei oder mehr Zählschleifen ineinander verschachtelt, so nennt man das eine **verschachtelte Zählschleife**.

Es gibt eine äußere Schleife mit der Zählvariablen j und eine innere Schleife mit der Zählvariablen i.

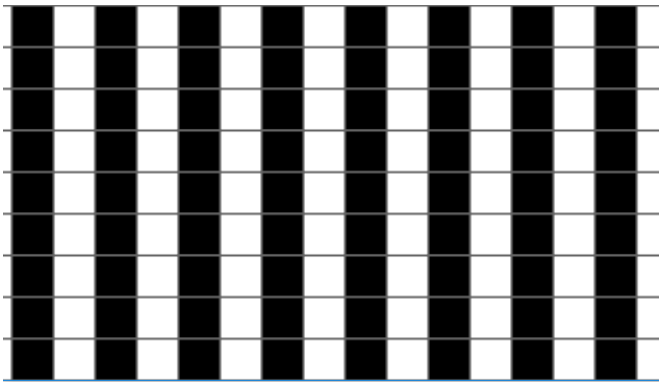
Die verschachtelte Zählschleife sieht jetzt folgendermaßen aus:

```
for (int j=0; j<=8; j++){
  for (int i=0; i<=7; i++){
    rect(10+2*s*i,5+j*s,s,s);
  }
}
```

14. *Schreibe eine neue Methode zeichneAlleQuadrate(), die diese verschachtelte Zählschleife nutzt. Teste dann dein Programm.*
15. *Überlege dir jetzt noch einmal ganz genau, was Schritt für Schritt passiert. Wenn es dir noch nicht klar ist, hole dir zur Hilfe das **Extrablatt** und fülle es Schritt für Schritt aus.*

Es wird neunmal nacheinander die innere Schleife aufgerufen, die jeweils acht Quadrate nebeneinander zeichnet.

Leider ist es jetzt keine optische Täuschung mehr, weil wir links immer nur 10 Pixel Abstand lassen.



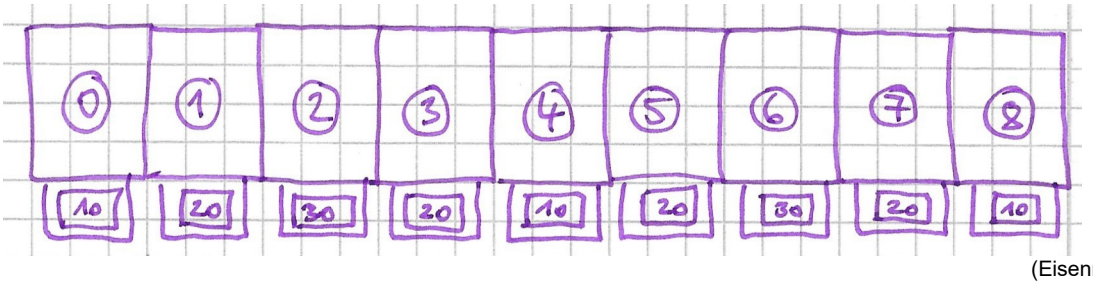
Statt dem Wert 10 als Abstand zum linken Rand, hätten wir gerne in jeder Reihe einen anderen Wert. Da sich der Wert mal erhöht, mal erniedrigt, können wir ihn nicht durch die Schleifenvariable j darstellen, wie es bei dem Abstand nach oben funktioniert hat.

Wir nutzen hier ein sogenanntes **Feld** (**Array**), das mehr als einen Wert gleichzeitig speichern kann. Du hast eine solche Datenstruktur schon im letzten Jahr kennen gelernt.

Zur Erinnerung:

Denken wir bei einer Variable an das Schachtelmodell (für jede Variable gibt es eine Schachtel, in der der aktuelle Wert der Variable liegt), können wir uns hier im Vergleich mehrere aneinandergeklebte Schachteln, die durchnummeriert sind, vorstellen.

In unserem Fall würde in den Schachteln jeweils der Abstand zum linken Rand als Wert liegen.



(Eisenmann)

In Java müssen wir das Feld bekannt machen und können es auch gleich mit den Werten füllen.

```
int[] abstaende = {10, 20, 30, 20, 10, 20, 30, 20, 10};
```

Die beiden eckigen Klammern nach dem Datentyp `int` zeigen an, dass `abstaende` ein Feld aus ganzen Zahlen ist.

Der Zugriff auf ein Element des Feldes geht auch mit eckigen Klammern: z.B. liefert `abstaende[0]` den Wert 10.

16. *Ändere dein Programm so ab, dass statt dem Abstand 10 der jeweilige Wert des Feldelementes genutzt wird (alte Zeile:* `rect(~~10~~+2*s*i,5+j*s,s,s);` *) Teste dein Programm und vergleiche mit der Lösung.*

VERSCHACHTELTE ZÄHLSCHLEIFE

```
for (int j=0; j<=8; j++){
  for (int i=0; i<=7; i++){
    rect(10+2*s*i,5+j*s,s,s);
  }
}
```

Schreibe hier Schritt für Schritt auf, was in der verschachtelten Zählschleife passiert und überlege dir auch, was Schritt für Schritt gezeichnet wird.

Äußere Schleife:	Innere Schleife:	
j = 0:	**i = 0:**	rect(10 + 2 * s * **0**, 5 + **0** * s, s, s), also rect(10, 5, s, s);
	i = 1:	rect(10 + 2 * s * **1**, 5 + **0** * s, s, s), also rect(10 + 2s, 5, s, s);
	i = 2:	
	i = 3:	
	i = 4:	
	i = 5:	
	i = 6:	
	i = 7:	
j = 1:	**i = 0:**	rect(10 + 2 * s * **0**, 5 + **1** * s, s, s), also rect(10, 5 + s, s, s);
	i = 1:	rect(10 + 2 * s * **1**, 5 + **1** * s, s, s), also rect(10 + 2s, 5 + s, s, s);
	i = 2:	
	…	
	i = 7:	
j = 2:	**i = 0:**	
	i = 1:	
	…	
	i = 7:	
…		
j = 8:	**i = 0:**	
	i = 1:	
	i = 2:	
	…	
	i = 7:	

Mögliche Lösungen:

Teil 1:

```
void parallelen(){ // zeichnet 10 graue Parallelen
  stroke(125); // Linienfarbe Graustufe
  for (int i=0; i<=9; i++){
    line(0,5+i*40, 640, 5+i*40);
  }
}

void setup(){
  size(640, 400);
  background(255); // Hintergrund weiß
  parallelen();  // Aufruf der Methode
}
```

Teil 1 mit Variable:

```
// globale Variablen
int s = 40; // Seitenlänge der Quadrate bzw. Abstand der Parallelen

void parallelen(){ // zeichnet 10 graue Parallelen
  stroke(125); // Linienfarbe Graustufe
  for (int i=0; i<=9; i++){
    line(0,5+i*s, 640, 5+i*s);
  }
}

void setup(){
  size(640, 400);
  background(255); // Hintergrund weiß
  parallelen();  // Aufruf der Methode
}
```

Teil 2 – Vorüberlegungen für Aufgabe 8:

Alle Quadrate sind gleich groß, d.h. sowohl Breite als auch Höhe haben den Wert s (Variable der Seitenlänge).

In der ersten Reihe haben auch alle Quadrate die gleiche y-Koordinate des Startpunktes. Wenn wir einen kleinen Abstand nach oben haben wollen, ist das z.B. 5.

Nur die x-Koordinate des Startpunktes ändert sich.

Quadrat 1 in der ersten Reihe: x = 10 (Vorgabe in der Aufgabe)

Das nächste Quadrat startet zwei Seitenlängen weiter rechts, da ja immer ein schwarzes Quadrat gezeichnet wird und dann ein Quadrat weiß bleibt.

Quadrat	1	2	3	4	
x-Koordinate	10	10 + 2*s	10+(2*s)*2	10+(2*s)*3	10+(2*s)*i
Zählvariable i	0	1	2	3	i

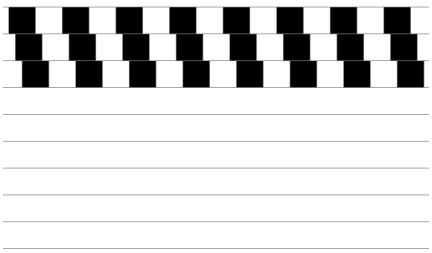
```
void zeichneQuadrate(){
  fill(0);  // Füllfarbe schwarz
  // fülle die erste Reihe mit Quadraten
  for (int i=0; i<=7; i++){
    rect(10+2*s*i,5,s,s);
  }
}
```

Teil 2:

```
void zeichneQuadrate(){
  fill(0);  // Füllfarbe schwarz
  // fülle die erste Reihe mit Quadraten
  for (int i=0; i<=7; i++){
    rect(10+2*s*i,5,s,s);
  }

  // fülle die zweite Reihe mit Quadraten
  for (int i=0; i<=7; i++){
    rect(20+2*s*i,5+s,s,s);
  }

  // fülle die dritte Reihe mit Quadraten
  for (int i=0; i<=7; i++){
    rect(30+2*s*i,5+2*s,s,s);
  }
}
```



Teil 3:

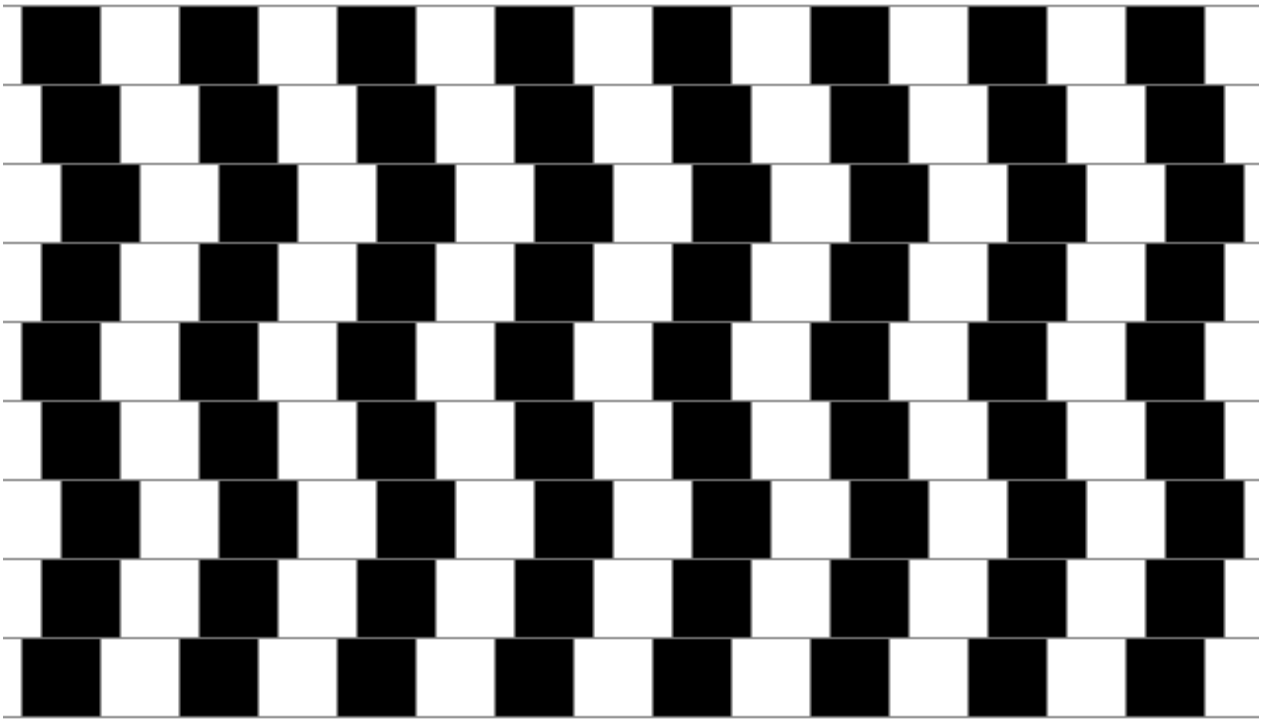
```
// globale Variablen
int s = 40; // Seitenlänge der Quadrate bzw. Abstand der Parallelen

int[] abstaende = {10, 20, 30, 20, 10, 20, 30, 20, 10}; // Abstände des 1. Quadrates nach links

void parallelen(){ // zeichnet 10 graue Parallelen
  stroke(125); // Linienfarbe Graustufe
  for (int i=0; i<=9; i++){
    line(0,5+i*s, 640, 5+i*s);
  }
}

void zeichneAlleQuadrate(){
  fill(0); // Füllfarbe schwarz
  for (int j=0; j<=8; j++){
    for (int i=0; i<=7; i++){
      // rect(10+2*s*i,5+j*s,s,s);  // Abstand bei allen Reihen gleich: d=10
      rect(abstaende[j]+2*s*i, 5+j*s, s, s);  // Abstand wird jeweils aus dem Feld gelesen
    }
  }
}

void setup(){
  size(640, 400);
  background(255); // Hintergrund weiß
  parallelen();  // Aufruf der Methode zum Zeichnen der Parallelen
  zeichneAlleQuadrate();
  //zeichneQuadrate(); // Aufruf der Methode zum Zeichnen der Quadrate
}
```



Optische Täuschungen – Weitere Übungen

Bei den folgenden beiden Bildern wird die optische Täuschung durch eine Nachbildwirkung auf der Netzhaut erzeugt.

Es handelt sich hier um sog. „Hermann-Gitter“.

Bild 1:

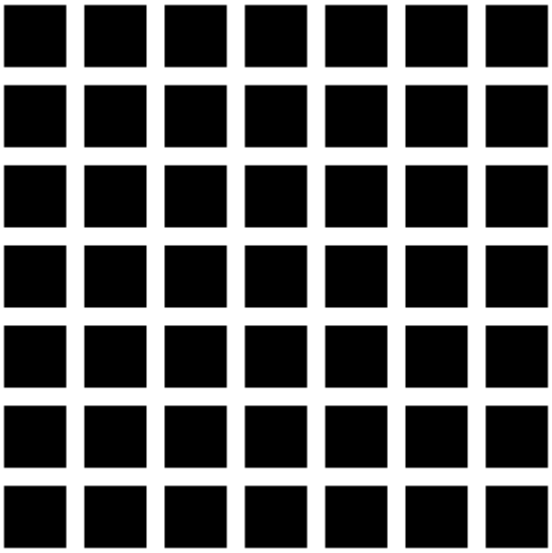


Bild: Screenshot von Ausführung des Programms „hermann_gitter_1“ (Eisenmann)

1. *Beschreibe, wie das Bild entsteht und was man zusätzlich sieht.*
2. *Speichere ein neues Programm unter „hermann_gitter_1“ ab.*
3. *Schreibe zunächst den Rahmen der setup-Methode, gib die Größe des Fensters (500x500) an und lasse den Hintergrund weiß „färben“.*
4. *Deklariere zwei globale Variablen: eine für die Seitenlänge der Quadrate und eine für den Abstand zwischen den Quadraten.*
5. *Schreibe eine Methode quadrate(), die die optische Täuschung zeichnet. Nutze dabei eine verschachtelte Zählschleife.*

Wenn du Hilfe brauchst, nutze die ausgelegten Hilfekarten.

Wenn du fertig bist, vergleiche dein Ergebnis mit der ausliegenden Lösung. Hole dir dann das nächste Blatt mit Bild 2.

Bild 2:

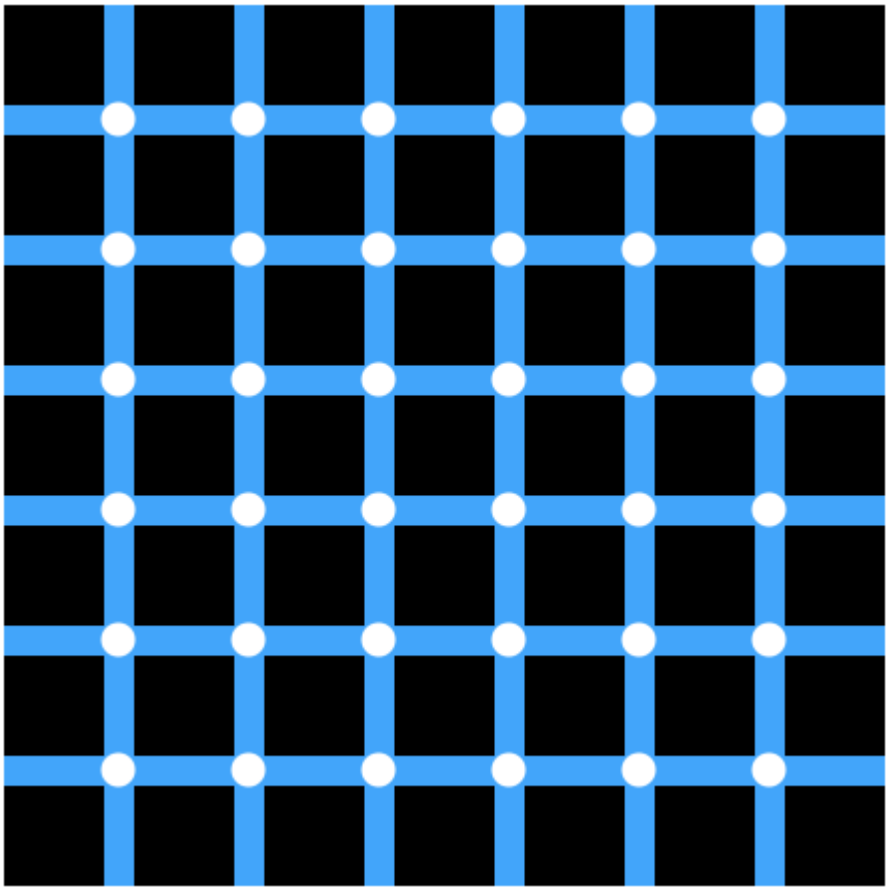


Bild: Screenshot von Ausführung des Programms „hermann_gitter_2“ (Eisenmann)

1. *Beschreibe, wie das Bild entsteht und was man zusätzlich sieht.*
2. *Speichere ein neues Programm unter „hermann_gitter_2“ ab.*
3. *Schreibe zunächst den Rahmen der setup-Methode, gib die Größe des Fensters (500x500) an und lasse den Hintergrund hellblau „färben“.*
4. *Deklariere wieder sinnvolle globale Variablen.*
5. *Schreibe eine Methode, die die optische Täuschung zeichnet. Nutze dazu verschachtelte Zählschleifen. Du kannst hier auch zwei Methoden schreiben: eine für die schwarzen Quadrate und eine für die weißen Kreise.*

Wenn du Hilfe brauchst, nutze die ausgelegten Hilfekarten.

Wenn du fertig bist, vergleiche dein Ergebnis mit der ausliegenden Lösung.

HILFEKARTEN ZU BILD 1

Tipp 1

Da der Hintergrund weiß ist, müssen nur die schwarzen Quadrate gezeichnet werden.

Überlege dir zunächst, wie du **eine** Reihe der Quadrate zeichnen kannst.

Tipp 2

In einer Reihe werden sieben gleiche Quadrate gezeichnet. Du kannst hier eine Zählschleife verwenden.

Versuche, die Zählschleife für die erste Reihe der Quadrate zu schreiben.

Tipp 3

Hat es geklappt? Dann schreibe jetzt die Zählschleife für die zweite Reihe. (Nächster Tipp: **6**)

Es hat nicht geklappt? Dann lies hier weiter:

Die Veränderung ist nur die Position der linken oberen Ecke der Quadrate.

Und hier bleibt die y-Koordinate gleich, nur die x-Koordinate verändert sich um die Seitenlänge eines Quadrates und den Abstand zwischen zwei Quadraten.

Tipp 4

y ist ein fester Wert, z.B. 30

x berechnet sich durch den Startwert (z.B. auch 30) + i * (seitenlaenge + abstand)

Ersetze dabei „seitenlaenge“ und „abstand“ durch den Namen deiner Variablen, die du eingeführt hast.

Tipp 5

```
for (int i=0; i<=6; i++) {
  rect(30 + i*(s + a), 30, s, s);
}
```

(s ist hier die Variable für die Seitenlänge, a für den Abstand)

Schreibe jetzt die Zählschleife für die zweite Reihe.

Tipp 6

Hast du es geschafft? Dann schreibe die Zählschleife für die dritte Reihe und überlege dir dabei, wie man alle Reihen mit einer weiteren Zählschleife programmieren könnte. (Nächster Tipp: **9**)

Hat es noch nicht geklappt? Dann lies hier weiter:

Was verändert sich in der zweiten Reihe im Vergleich zur ersten?

Was bleibt gleich?

Tipp 7

Es bleibt alles gleich bis auf die y-Koordinate.

Hier wird zum Startwert (30) einmal die Seitenlänge und einmal der Abstand addiert.

Tipp 8

Die Schleife für die zweite Reihe sieht folgendermaßen aus:

```
for (int i=0; i<=6; i++) {
  rect(30 + i*(s + a), 30 + s + a, s, s);
}
```

Schreibe jetzt die Zählschleife für die dritte Reihe und überlege dir dabei, wie man alle Reihen in einer weiteren Zählschleife programmieren könnte.

Tipp 9

Mögliche Lösung für die dritte Reihe:

```
for (int i=0; i<=6; i++) {
  rect(30 + i*(s + a), 30 + 2* (s + a), s, s);
}
```

Tipp 10

Hast du herausgefunden, was sich von Reihe zu Reihe ändert und wie du das mit einer weiteren Zählschleife realisieren kannst? - Dann vergleiche dein Ergebnis mit der Lösung.

Du hast es noch nicht geschafft? Dann schau dir mal die y-Koordinaten an:

30 30 + s + a 30 + 2 * (s + a) 30 + 3 * (s + a), usw.

Wie könnte man das allgemein mit einer neuen Zählvariable j beschreiben?

Tipp 11

Startet j bei 0, dann kann man die y-Koordinate durch 30 + j * (s + a) beschreiben.

Schreibe jetzt die zweite Zählschleife um die erste herum und vergleiche dein Ergebnis mit der Lösung.

Lösung

```
// globale Variablen
int s = 30; // Seitenlaenge Quadrat
int a = 10; // Abstand Quadrate

// Methode, die optische Täuschung zeichnet
void quadrate() {
  fill(0); // Füllfarbe schwarz
  for (int j=0; j<=6; j++) {
    for (int i=0; i<=6; i++) {
      rect(30+i*(s+a), 30+j*(s+a), s, s);
    }
  }
}

// setup-Methode
void setup() {
  size(500, 500);
  background(255); // Hintergrund weiß
  quadrate();
}
```

HILFEKARTEN ZU BILD 2

Tipp 1

Um die schwarzen Quadrate zu zeichnen, kannst du die verschachtelte Zählschleife aus dem Programm zu **Bild 1** nutzen.

Überlege dir, was dann noch fehlt.

Tipp 2

An den Kreuzungspunkten müssen kleine weiße Kreise gezeichnet werden.

Überlege dir, wo die Mittelpunkte liegen und wie groß etwa der Durchmesser sein muss.

Tipp 3

(Variable für die Seitenlänge: s, Variable für den Abstand: a, Variable für Abstand zum Rand: start)

Der x-Koordinate des Mittelpunktes des ersten Kreises ist start + s + a / 2. (Achtung: Das Zeichen / steht für „geteilt durch“)

Die y-Koordinate ist start + s + a / 2.

Der Durchmesser (also Breite und Höhe der Ellipse) ist ein bisschen größer als der Abstand a.

Was verändert sich nach rechts? Was nach unten?

Tipp 4

Nach rechts verändert sich die x-Koordinate um s + a.

Nach unten die y-Koordinate um s + a.

Der Durchmesser bleibt gleich.

Schreibe die Zählschleife für die obere Reihe.

Tipp 5

Eine mögliche Lösung:

```
for (int i=0; i<=5; i++) {
  ellipse(start+s+a/2+i*(s + a), start+s+a/2, a+2, a+2);
}
```

Schreibe jetzt die Zählschleife, die alle Reihen zeichnet.

Tipp 6

Hast du es geschafft? Dann vergleiche dein Ergebnis mit der Lösung.

Hat es noch nicht geklappt?

Dann schreibe zunächst eine Zählschleife für die zweite Reihe, die dritte, … So lange, bis du erkennst, was sich verändert.

Tipp 7

Die Veränderung ist in der y-Koordinate.

Mit einer neuen Zählvariable j, die bei 0 startet, lässt sich die y-Koordinate durch start + s + a/2 + j * (s + a) beschreiben.

Schreibe jetzt die verschachtelte Zählschleife fertig. Vergleiche dann dein Ergebnis mit der Lösung.

Lösung

```
// globale Variablen
int s = 50; // Seitenlaenge Quadrat
int a = 15; // Abstand Quadrate
int start = 30; // Abstand zum Rand

// Methode, die schwarze Quadrate zeichnet
void quadrate() {
  fill(0);
  for (int i=0; i<=6; i++) {
    for (int j=0; j<=6; j++) {
      rect(start+j*(s+a), start+i*(s+a), s, s);
    }
  }
}

// Methode, die weiße Kreise zeichnet
void kreise() {
  fill(255);
  noStroke(); // keine Umrandung
  for (int j=0; j<6; j++) {
    for (int i=0; i<6; i++) {
      ellipse(start+s+a/2+(s+a)*i, start+s+a/2+(s+a)*j, a+2, a+2);
    }
  }
}

// setup-Methode
void setup() {
  size(500, 500);
  background(66, 165, 250);  // Hintergrundfarbe hellblau
  quadrate();
  kreise();
}
```

Weitere optische Täuschungen

Du hast schon die sog. Kaffeehaus-Täuschung kennen gelernt und selbst programmiert. Es gibt noch weitere optische Täuschungen, die das Gefühl vermitteln, dass parallele Geraden krumm verlaufen.

Beispiel 1:

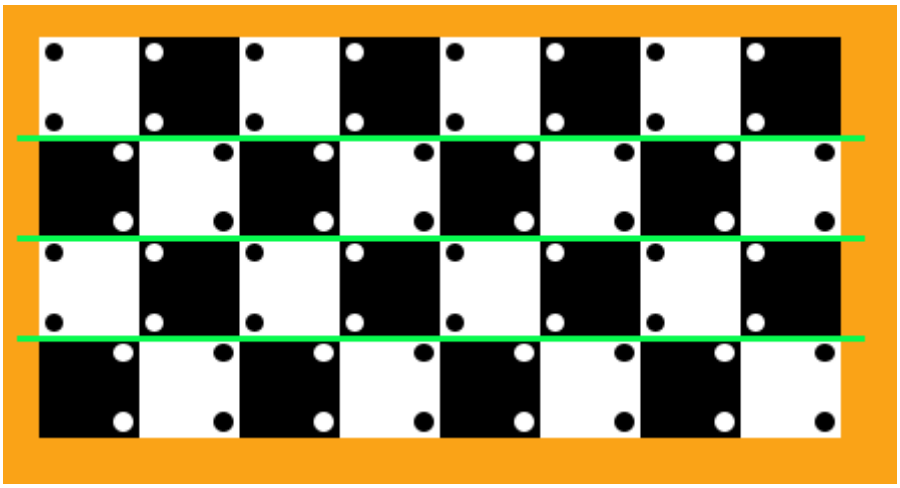
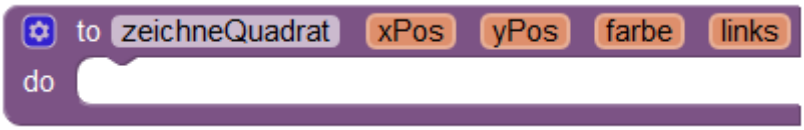


Bild: Screenshot von Ausführung des Programms „wellen1_mit_farbwechsel“ (Eisenmann)

Das Zeichnen des Bildes wäre mit dem, was du bisher in Java gelernt hast, einfach zu programmieren, wenn die kleinen Kreise in den Quadraten nicht wären. Auch diese könnten wir mit Schleifen extra zeichnen lassen, das wäre aber ziemlich umständlich.

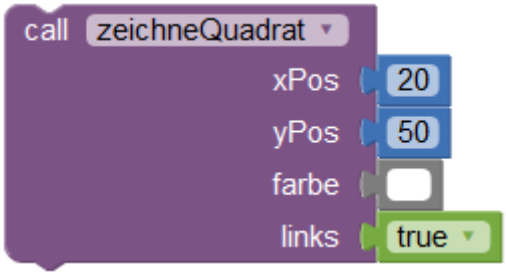
In Scratch oder dem MIT-App-Inventor hast du **Unterprogramme** kennen gelernt. Wir haben Codeteile, die wir häufiger gebraucht haben, zusammengefasst.

Zur Erinnerung:



Das Unterprogramm hier heißt „zeichneQuadrat“ und es können ihm vier Werte übergeben werden. Die Koordinaten der linken oberen Ecke, die Farbe und die Information, ob die Kreise links (true) oder rechts (false) liegen.

Der Aufruf des Unterprogramms könnte z.B. folgendermaßen aussehen:



Unterprogramme gibt es auch in Java. Man nennt sie auch **Methoden**. Wir haben schon einige fertige Methoden genutzt: `ellipse, rect, fill`, usw. Außerdem haben wir schon einfache Methoden selbst geschrieben.

Um eine Methode zu schreiben, müssen wir uns zunächst überlegen, ob einfach etwas ausgeführt werden soll, oder ob wir etwas ausführen lassen und einen Rückgabewert erwarten.

Allgemein sieht der Rahmen einer **Methode** folgendermaßen aus:

```
<Typ Rückgabewert> <Name der Methode> (<mögliche Übergabeparameter>)
{

 … }
```

Wird keine Rückgabe erwartet, steht vor dem Namen der Methode `void`.

Gibt es keine Übergabeparameter, so bleibt die Klammer leer.

1. *Lies dir das Blatt „Infos zu Methoden in Java“ durch, wenn du es noch nicht bei den Übungen zur Zählschleife gemacht hast.*

Bei der optischen Täuschung von oben möchten wir einfach etwas ausführen lassen (das Zeichnen eines Quadrates mit zwei Punkten), es gibt also keinen Rückgabewert. Um ein Quadrat zeichnen zu können, brauchen wir die Position (Koordinaten der linken oberen Ecke), die Hintergrundfarbe und die Information, ob die Kreise links oder rechts liegen.

Die benötigten Typen hast du schon kennen gelernt: `int` für eine ganze Zahl (auch Farben werden durch ganze Zahlen dargestellt) und `boolean` für die Wahrheitswerte `true` und `false`.

Der Rahmen unserer Methode sieht also folgendermaßen aus:

```
void zeichneQuadrat(int x, int y, int farbe, boolean links) {

}
```

(Die Variablen werden in der Entwicklungsumgebung übrigens unterkringelt, weil sie in der Methode noch nicht genutzt wurden.)

2. *Speichere eine neue Datei unter „wellen1“ ab.*
3. *Schreibe dann zunächst den Rahmen der setup()-Methode.*
4. *Schreibe den Rahmen der eigenen Methode „zeichneQuadrat“ (s.o.).*

Noch sind beide Methoden leer.

5. *Überlege dir mit deiner Sitznachbarin / deinem Sitznachbarn, was in der Methode „zeichneQuadrat“ passieren soll.*

Habt ihr es herausgefunden? Dann holt euch das nächste Blatt.

Es soll ein Quadrat an der vorgegebenen Position mit der vorgegebenen Farbe (weiß oder schwarz) gezeichnet werden.

In dem Quadrat sollen zwei kleine Kreise in der jeweils anderen Farbe entweder links oder rechts gezeichnet werden. Je nachdem, ob `true` (für links) oder `false` (für rechts) übergeben wird.

Um ein Quadrat zeichnen zu können, fehlt uns noch die Seitenlänge.

6. *Deklariert und initialisiert eine globale Variable s für die Seitenlänge.*

Euer Code sollte bisher etwa so aussehen:

```
// globale Variable
int s = 40; // Seitenlänge der Quadrate

void zeichneQuadrat(int x, int y, int farbe, boolean links) {

}

void setup() {

}
```

Bevor wir in der Methode zeichneQuadrat() ein Quadrat zeichnen lassen, müssen wir die Füllfarbe festlegen.

Dann wird die Methode für das Zeichnen eines Rechtecks mit den nötigen Werten aufgerufen.

```
void zeichneQuadrat(int x, int y, int farbe, boolean links) {
  fill(farbe); // der übergebene Wert der Variable farbe wird als Füllfarbe übergeben
  rect(x, y, s, s); // ein Quadrat mit Seitenlänge s wird an der Position (x|y) gezeichnet
}
```

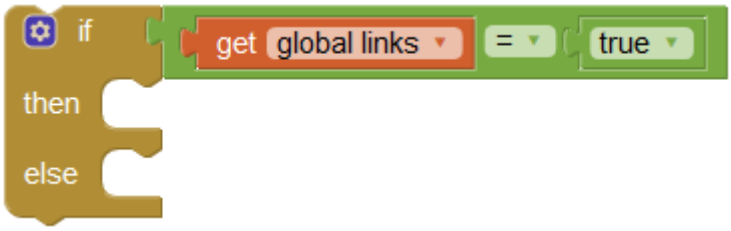
7. *Ergänze dein Programm entsprechend.*
   Du hast gelernt, eine Methode, die du selbst geschrieben hast, in setup-Methode aufzurufen, um sie zu testen. Das kannst Du hier beispielsweise mit dem Befehl zeichneQuadrat(50,100,255,true);
   tun. Hierbei zeichnest du ein Rechteck mit der linken oberen Ecke (50|100) mit der Füllfarbe weiss (255). Das „true“ wird an dieser Stelle noch nicht verwendet.

Jetzt fehlen noch die beiden kleinen Kreise auf der linken oder rechten Seite. Auf welcher Seite sie gezeichnet werden sollen, legt der Übergabeparameter `links` fest. Beim Aufruf der Methode wird entweder `true` oder `false` übergeben. Welche Farbe sie haben, hängt von der Farbe des Quadrates ab.

8. *Ergänze den folgenden Satz für unsere Abbildung: „Falls die Farbe des Quadrates weiß ist, ist die Füllfarbe der Kreise …, sonst … .“*
9. *Ergänze den folgenden Satz: „Falls die Variable `links` den Wert `true` besitzt, dann …, sonst … .“*
10. *Welche Kontrollstruktur brauchst du hier jeweils?*

Wir brauchen eine **Verzweigung**.

Zur Erinnerung:



Falls die Bedingung erfüllt ist (hier: Variable `links` besitzt den Wahrheitswert `true`), wird ausgeführt, was hinter `then` steht, falls nicht, das was hinter `else` steht.

Eine **Verzweigung** in Java sieht folgendermaßen aus:

```
if (<Bedingung>) {
  …
} else {
  …
}
```

In unserem Fall:

```
if (farbe == 255) {
  // Quadrat ist weiß, Füllfarbe muss schwarz werden
} else {
  // Quadrat ist nicht weiß (schwarz), Füllfarbe muss weiß werden
}
```

oder:

```
if (links == true){
  // zeichne die beiden Kreise links
} else {
  // zeichne die beiden Kreise rechts
}
```

Achtung: Für eine Zuweisung genügt ein einfaches Gleichheitszeichen. Möchte man in der Bedingung aber auf Gleichheit überprüfen wie hier, so muss man zwei Gleichheitszeichen schreiben.

11. *Schreibe zunächst in deine Methode zeichneQuadrat(…) die erste Verzweigung zur Farbauswahl der kleinen Kreise. Vergleiche, wenn nötig, mit der ausliegenden Lösung.*
12. *Für die zweite Verzweigung, die sich um das Zeichnen der Kreise kümmern soll, ist eine Skizze (auf Papier) sehr hilfreich. Fertige eine solche Skizze eines Quadrates mit zwei kleinen Kreisen innerhalb an. Überlege dir dabei, welche Werte die Methode ellipse() jeweils übergeben bekommen muss. Nutze des Modus ellipseMode(CORNERS)[1].*

Weitere Aufträge findest du auf dem nächsten Blatt.

1 ellipse(x1,y1,x2,y2) – (x1|y1) linke obere u. (x2|y2) rechte untere Ecke des umschließenden Rechtecks

13. *Schreibe jetzt mithilfe deiner Skizze die Verzweigung, in der die beiden kleinen Kreise gezeichnet werden. (In der vorliegenden Lösung wurde als Kreisdurchmesser ein Viertel der Seitenlänge des Quadrates gewählt und der Einfachheit halber wurden die Kreise zunächst direkt an den Quadratrand gesetzt.)*
14. *Teste dein Programm, indem du in der setup()-Methode zunächst die Größe des Fensters (z.B. 600 auf 400 Pixel) einstellst und dann die Methode „zeichneQuadrat()“ mit verschiedenen Werten aufrufst.*
15. *Vergleiche deinen Programmcode mit der ausliegenden Lösung.*

16. *Schreibe jetzt den Rahmen der Methode alleQuadrate(), in der dann die optische Täuschung gezeichnet werden soll.*
17. *Nutze eine Zählschleife, um die erste Reihe von Quadraten zu zeichnen. Achte darauf, dass von Quadrat zu Quadrat die Füllfarbe wechselt, die Position der Kreise aber gleich bleibt. Nutze dazu eine Variable* `aktuellefarbe` *vom Typ* `int`*.*
18. *Programmiere auch das Zeichnen der zweiten, dritten und vierten Reihe.*
19. *Vergleiche mit der ausliegenden Lösung.*

20. ***** Hast du eine Idee, wie wir **alle** Reihen mithilfe einer verschachtelten Zählschleife zeichnen lassen können? Besprich dich mit deiner Sitznachbarin / deinem Sitznachbarn und probiert eure Ideen aus.*

Weitere Aufgaben:

21. *Programmiert das Zeichnen der grünen Trennlinien zwischen den Reihen.*
22. *Führt eine Variable a für den Abstand der Kreise zum Quadratrand ein und sucht einen Wert, für den die optische Täuschung am wirkungsvollsten ist.*
23. *Verändert euer Programm so, dass die optische Täuschung Abstand zu den Rändern hat.*
24. *Wir haben mehrere Male den Wechsel der Farbe von schwarz auf weiß und umgekehrt benutzt. Schreibe eine Methode „farbwechsel“, der man die Farbe schwarz (0) oder weiß (255) übergeben kann und die die jeweils andere Farbe zurückgibt.*
25. *Nutze die Methode überall dort in deinem Programm, wo ein Farbwechsel stattfindet.*
26. *Teste dein Programm und vergleiche dann mit der ausliegenden Lösung.*

Der MIT App Inventor (*http://appinventor.mit.edu*) wurde ursprünglich von einem Entwicklerteam um Mark Friedman und Hal Abelson bei Google entwickelt und 2012 an das MIT übergeben.

Der MIT App Inventor wird unter der Creative Commons Attribution-ShareAlike 3.0 Unported License veröffentlicht: *https://creativecommons.org/licenses/by-sa/3.0*

Lösung nach Aufgabe 11:

```
void zeichneQuadrat(int x, int y, int farbe, boolean links) {
  fill(farbe); // der übergebene Wert der Variable farbe wird als Füllfarbe übergeben
  rect(x, y, s, s); // ein Quadrat mit Seitenlänge s wird an der Position (x|y) gezeichnet

  // Farbauswahl Kreise
  if (farbe == 255) {  // Quadrat ist weiß
    fill(0); // Füllfarbe schwarz wird gewählt
  } else {  // Quadrat ist nicht weiß (schwarz)
    fill(255);  // Füllfarbe weiß wird gewählt
  }
}
```

Lösung nach Aufgabe 14:

```
// globale Variable
int s = 40; // Seitenlänge der Quadrate

void zeichneQuadrat(int x, int y, int farbe, boolean links) {
  fill(farbe); // der übergebene Wert der Variable farbe wird als Füllfarbe übergeben
  rect(x, y, s, s); // ein Quadrat mit Seitenlänge s wird an der Position (x|y) gezeichnet

  // Farbauswahl Kreise
  if (farbe == 255) {  // Quadrat ist weiß
    fill(0); // Füllfarbe schwarz wird gewählt
  } else {  // Quadrat ist nicht weiß (schwarz)
    fill(255);  // Füllfarbe weiß wird gewählt
  }

  // Zeichnen der beiden kleinen Kreise
  ellipseMode(CORNERS);
  noStroke();
  int d = s/4;
  if (links == true) {
    ellipse(x, y, x+d, y+d);  // links oben
    ellipse(x, y + 3*d, x+d, y+s); // links unten
  } else {
    ellipse(x + 3*d, y, x+s, y+d);  // rechts oben
    ellipse(x + 3*d, y + 3*d, x+ s, y+s);  // rechts unten
  }
}

void setup() {
  size(600, 400);
  zeichneQuadrat(100, 100, 0, true);
  zeichneQuadrat(100, 200, 255, false);
}
```

Lösung nach Aufgabe 19:

```
void alleQuadrate() {
  noStroke();
  // erste Reihe
  int aktuellefarbe = 255; // Farbe des ersten Quadrates ist weiß
  for (int i=0; i<=7; i++) {
    zeichneQuadrat(i*s, 0, aktuellefarbe, true);
    // Farbwechsel für nächstes Quadrat
    if (aktuellefarbe == 255) {
      aktuellefarbe = 0;
    } else {
      aktuellefarbe = 255;
    }
  }
  // zweite Reihe
  aktuellefarbe = 0; // Farbe des ersten Quadrates ist schwarz
  for (int i=0; i<=7; i++) {
    zeichneQuadrat(i*s, s, aktuellefarbe, false);
    // Farbwechsel für nächstes Quadrat
    if (aktuellefarbe == 255) {
      aktuellefarbe = 0;
    } else {
      aktuellefarbe = 255;
    }
  }
    // dritte Reihe
  aktuellefarbe = 255; // Farbe des ersten Quadrates ist weiß
  for (int i=0; i<=7; i++) {
    zeichneQuadrat(i*s, 2*s, aktuellefarbe, true);
    // Farbwechsel für nächstes Quadrat
    if (aktuellefarbe == 255) {
      aktuellefarbe = 0;
    } else {
      aktuellefarbe = 255;
    }
  }
  // vierte Reihe
  aktuellefarbe = 0; // Farbe des ersten Quadrates ist schwarz
  for (int i=0; i<=7; i++) {
    zeichneQuadrat(i*s, 3*s, aktuellefarbe, false);
    // Farbwechsel für nächstes Quadrat
    if (aktuellefarbe == 255) {
      aktuellefarbe = 0;
    } else {
      aktuellefarbe = 255;
    }
  }
}
```

Lösung nach Aufgabe 26:

```
// globale Variable
int s = 40; // Seitenlänge der Quadrate
int a = 2; // Abstand Kreise - Quadratrand
int d = 20; // Abstand der optischen Täuschung zum Rand

int farbwechsel (int farbe) {
  if (farbe == 255) { // Farbe ist weiß
    farbe = 0;
  } else {  // Farbe ist nicht weiß
    farbe = 255;
  }
  return farbe;
}


void zeichneQuadrat(int x, int y, int farbe, boolean links) {
  fill(farbe); // der übergebene Wert der Variable farbe wird als Füllfarbe übergeben
  rect(x, y, s, s); // ein Quadrat mit Seitenlänge s wird an der Position (x|y) gezeichnet

  // Farbauswahl Kreise
  fill(farbwechsel(farbe));

  // Zeichnen der beiden kleinen Kreise
  ellipseMode(CORNERS);
  noStroke();
  int d = s/4;
  if (links == true) {
    ellipse(x+a, y+a, x+d+a, y+d+a);  // links oben
    ellipse(x+a, y + 3*d-a, x+d+a, y+s-a); // links unten
  } else {
    ellipse(x + 3*d-a, y+a, x+s-a, y+d+a);  // rechts oben
    ellipse(x + 3*d-a, y + 3*d-a, x+ s-a, y+s-a);  // rechts unten
  }
}
```

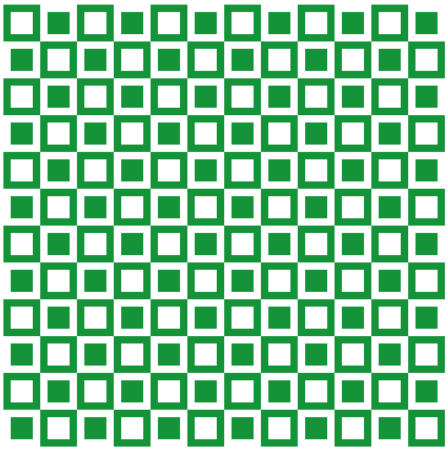
```
void alleQuadrate() {
  noStroke();
  int aktuellefarbe = 255; // Farbe des ersten Quadrates ist weiß
  boolean pos_aktuell = true; // Kreise sind zunächst links
  for (int j=0; j<=3; j++) {
    for (int i=0; i<=7; i++) {
      zeichneQuadrat(d + i*s, d + j*s, aktuellefarbe, pos_aktuell);
      // Farbwechsel für nächstes Quadrat
      aktuellefarbe = farbwechsel(aktuellefarbe);
    } // end Zeichnen einer Reihe
    // Wechsel Startfarbe
    aktuellefarbe = farbwechsel(aktuellefarbe);
    // Wechsel Position der Kreise
    if (pos_aktuell == true) {
      pos_aktuell = false;
    } else {
      pos_aktuell = true;
    }
  }

  //zeichne die drei Linien zwischen die Quadrate
  for (int i=0; i<=2; i++) {
    stroke(8, 250, 78);  // Linienfarbe hellgrün
    strokeWeight(2);
    line(d, d + s + s*i, d + 8*s, d + s + s*i);
  }
}

void setup() {
  size(600, 400);
  // Zeichnen der optischen Täuschung
  alleQuadrate();
}
```

Weitere Aufgaben

Beispiel 1:



Bilder: Screenshots von Ausführung des Programms „wellen_b1_lsg_farbwechsel“ (Eisenmann)

1. *Programmiere das Zeichnen der optischen Täuschung aus Beispiel 1. Verwende dazu alles, was du gelernt hast: eigene Methoden (zeichneQuadrat(…) und farbwechsel(...)), verschachtelte Zählschleife. (Hilfekarten liegen aus.)*
2. *In einer zweiten Version sollen die Quadrate das ganze Fenster füllen. Dazu brauchst du eine neue Schleifenart. Hol dir dazu die Anleitung bei deiner Lehrerin.*
3. *Suche dir eine weitere optische Täuschung aus, die du schon programmiert hast, speichere sie unter einem neuen Namen ab und verändere sie mithilfe der neuen Schleifenart so, dass sie das ganze Fenster füllt.*
4. ***** Eine Herausforderung ist Beispiel 2, wenn du noch Zeit hast. Ein paar Hinweise dazu liegen aus.*

Beispiel 2 *:**

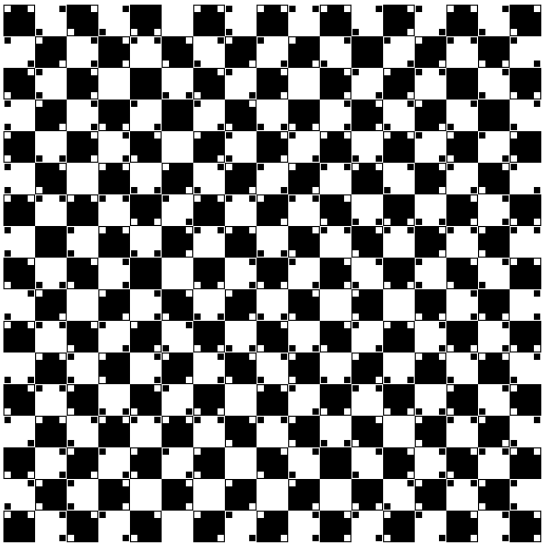


Bild: Screenshot von Ausführung des Programms „wellen2_zufall“ (Eisenmann)

HILFEKARTEN ZU BEISPIEL 1

Tipp 1

Überlege dir wieder eine Methode `zeichneQuadrat()`, mit der du ein Quadrat mit grünem Rand und weißer Füllung oder weißem Rand und grüner Füllung zeichnen lassen kannst.

Nutze zwei globale Variablen für die Farben `weiss` und `gruen`. Wähle einen Grünton aus der Farbauswahl unter Tools.

Es ist später einfacher, die Methode zu nutzen, wenn man zwei Quadrate übereinander zeichnen lässt. Z.B. ein grünes Quadrat mit Seitenlänge s und darauf ein weißes Quadrat, das überall vom Rand den Abstand d hat.

Tipp 2

Hast du es geschafft? Dann vergleiche mit der Lösung auf Tippkarte 5.

Du hast noch keine Idee? Dann überlege dir zunächst, was du der Methode mitteilen willst, also welche Übergabeparameter du brauchst.

Tipp 3

Hier genügen im Gegensatz zu der vorherigen optischen Täuschung die Position der linken oberen Ecke und die Grundfarbe. Hier kannst du entscheiden, ob du die Farbe des hinteren oder die des darüber liegenden Quadrates übergeben willst.

Tipp 4

Der Rahmen deiner Methode könnte jetzt z.B. so aussehen:

```
void zeichneQuadrat(int x, int y, int rahmenfarbe) { … }
```

Schreibe jetzt die Methode so, dass das Quadrat an der Position mit der Farbe gezeichnet wird.

Tipp 5

```
void zeichneQuadrat(int x, int y, int rahmenfarbe) {

  fill(rahmenfarbe); // Linienfarbe einstellen
  noStroke();
  rect(x, y, s, s); // Zeichnen des hinteren Quadrates

  // Bestimmen der Farbe des inneren Quadrates
  if (rahmenfarbe == weiss) {
    fill(gruen);
  } else {
    fill(weiss);
  }

  rect(x+d, y+d, s-2*d, s-2*d); // Zeichnen des inneren Quadrates
}
```

Teste die Methode, indem du sie zweimal aufrufst. Nutze zwei unterschiedliche Positionen und übergib einmal die Farbe weiss, einmal gruen.

Tipp 6

Dein Test könnte etwa so aussehen:

```
// globale Variable
int s=50; // Seitenlänge der Quadrate
int d=5; // Abstand inneres Quadrat zum Rand
int weiss = 255;
int gruen = #1CAF38;


// setup-Methode
void setup() {
  size(700, 400);
  background(255); // weißer Hintergrund

  // Test
  zeichneQuadrat(30, 30, gruen);
  zeichneQuadrat(30+s, 30, weiss);
}
```



Tipp 7

Überlege dir jetzt, wie du die erste Reihe mit einer Zählschleife programmiere kannst und in einem zweiten Schritt mit einer weiteren Zählschleife alle Reihen.

Tipp 8

Hast du es allein geschafft? Dann vergleiche mit der Lösung.

Du brauchst Hilfe? Dann rufe zunächst deine Methode für die ersten drei Quadrate auf und überlege dir, was sich verändert.

Schreibe jetzt die Zählschleife für die erste Reihe.

Tipp 9

Es ändert sich wieder nur die x-Koordinate der Startposition der Quadrate. Es kommen jedes Mal s Pixel dazu.

Der Code der Zählschleife könnte so aussehen:

```
void alleQuadrate(){
  int farbe = gruen; // Festlegen der ersten Rahmenfarbe
  // erste Reihe
  for (int i=0; i<12; i++) {
    zeichneQuadrat(30+i*s, 30, farbe);
    if (farbe == gruen) {  // du könntest hier wieder eine Methode Farbwechsel schreiben
      farbe = weiss;
    } else {
      farbe = gruen;
    }
  }
}
```

Es ist hier ein Abstand zum Fensterrand mit 30 Pixel angegeben. Das Bild dazu sieht so aus:



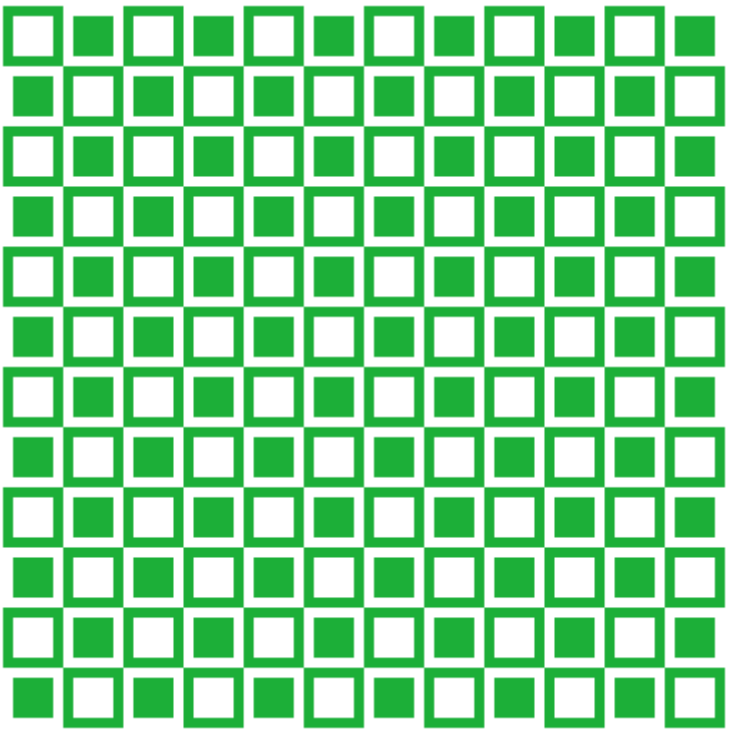
Wenn du noch keine Idee hast, schreibe den Code für die zweite Reihe,… Überlege dir dann, was sich von Reihe zu Reihe verändert und schreibe die zweite Zählschleife für die Reihen.

Vergleiche dann mit der Lösung.

Lösung

```
void alleQuadrate(){
  int farbe = gruen; // Festlegen der ersten Rahmenfarbe
  for (int j=0; j<12; j++) { // j zählt die Reihen

    for (int i=0; i<12; i++) {  // i zählt die Quadrate in einer Reihe
      zeichneQuadrat(30+i*s, 30+j*s, farbe);
      if (farbe == gruen) {  // du könntest hier wieder eine Methode Farbwechsel schreiben
        farbe = weiss;
      } else {
        farbe = gruen;
      }
    }
    // Wechsel der Farbe des ersten Quadrates für die nächste Reihe
    if (farbe == gruen) {  // du könntest hier wieder eine Methode Farbwechsel schreiben
      farbe = weiss;
    } else {
      farbe = gruen;
    }
  }
}
```



Schreibe jetzt noch eine Methode Farbwechsel und nutze sie geeignet, wenn du es noch nicht gemacht hast.

Lösung mit Farbwechsel

```
// globale Variable
int s=30; // Seitenlänge der Quadrate
int d=5; // Abstand inneres Quadrat zum Rand
int weiss = 255;
int gruen = #1CAF38;


// setup-Methode
void setup() {
  size(700, 700);
  background(255); // weißer Hintergrund
  alleQuadrate();
}

int farbwechsel(int farbe) {
  if (farbe == weiss) {  // Wechsel von weiß auf grün
    return gruen;
  } else {  // Wechsel von grün auf weiß
    return weiss;
  }
}

void zeichneQuadrat(int x, int y, int rahmenfarbe) {

  fill(rahmenfarbe); // Linienfarbe einstellen
  noStroke();
  rect(x, y, s, s); // Zeichnen des hinteren Quadrates

  // Bestimmen der Farbe des inneren Quadrates
  fill(farbwechsel(rahmenfarbe));

  rect(x+d, y+d, s-2*d, s-2*d); // Zeichnen des inneren Quadrates
}

void alleQuadrate(){
  int farbe = gruen; // Festlegen der ersten Rahmenfarbe
  for (int j=0; j<12; j++) { // j zählt die Reihen

    for (int i=0; i<12; i++) {  // i zählt die Quadrate in einer Reihe
      zeichneQuadrat(30+i*s, 30+j*s, farbe);
      farbe = farbwechsel(farbe);
    }
    // Wechsel der Farbe des ersten Quadrates für die nächste Reihe
    farbe = farbwechsel(farbe);
  }
}
```

Lösung mit while-Schleife:

(Rest siehe Lösung mit Farbwechsel)

```
void alleQuadrate(){
  int farbe = gruen; // Festlegen der ersten Rahmenfarbe
  int pos_x = 0;
  int pos_y = 0;
  while (pos_y < 700) { // unterer Rand ist noch nicht erreicht

    while (pos_x < 700) {  // rechter Rand ist noch nicht erreicht
      zeichneQuadrat(pos_x, pos_y, farbe);
      farbe = farbwechsel(farbe);
      pos_x = pos_x + s;  // verändern der x-Position um s
    }
    // Wechsel der Farbe des ersten Quadrates für die nächste Reihe
    farbe = farbwechsel(farbe);
    // x-Position wieder an linken Rand setzen
    pos_x = 0;
    pos_y = pos_y + s; // verändern der y-Position um s
  }
}
```

HINWEISE UND TIPPS ZU BEISPIEL 2

Information

Diese optische Täuschung gibt es in vielen Varianten. Es gibt solche, bei denen bestimmte Quadrate eine feste Position haben und solche, bei denen die verschiedenen Quadrate durch Zufall bestimmt werden.

Beispiel mit fester Position:

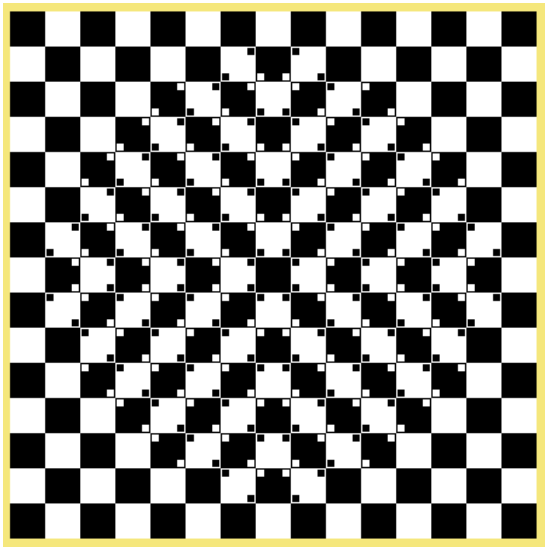


Bild: Screenshot von Ausführung des Programms „wellen2_fest“ (Eisenmann)

Du kannst selbst wählen, welche Art du programmieren möchtest.

Ganzzahlige **Zufallszahlen** zwischen a und b werden durch die Methode `random(a,b)` erzeugt. Allerdings gibt die Methode nicht den Typ `int`, sondern den Typ `float` zurück. Der steht für Kommazahlen.

Um einen Wert vom Typ int zu erhalten, kann man die Methode `round()` nutzen, die den übergebenen Wert auf eine ganze Zahl rundet.

Für festgelegte Positionen könntest du wieder ein Feld (`array`) nutzen, in dem du die Art der Quadrate vorab festlegst.

Wenn du keine Idee hast, wie man die **Position der kleinen Quadrate** übergeben kann, hier zwei Möglichkeiten:

1. Nutze vier Variablen vom Typ `boolean`. Ist das kleine Quadrat sichtbar, besitzt es den Wert `true`, wenn nicht, den Wert `false`.

2. Du könntest die Position der kleinen Quadrate durch Binärzahlen darstellen. Reihenfolge z.B. im Uhrzeigersinn von linker oberer Ecke bis zu linker unterer Ecke. Ist ein Quadrat sichtbar, hat es den Wert 1, sonst den Wert 0.

Kein Quadrat wäre die 0000b, alle vier Quadrate wäre die 1111b, z.B. Quadrat links oben und rechts unten wäre 1010b, usw.

Eine neue Schleifenart

Bei unseren bisherigen Beispielen wussten wir vorab immer, wie oft etwas wiederholt werden sollte. Wir konnten deshalb Zählschleifen nutzen.

Zur Erinnerung:

```
for (int i=0; i<10; i++) {
  // i läuft von 0 bis 9, es wird also 10-mal wiederholt
}
```

Wir könnten für zehn Wiederholungen auch i von 1 bis 10, von 2003 bis 2012, etc. laufen lassen. Welchen Bereich wir wählen, entscheiden wir je nachdem, wie wir die Zählvariable i in der Schleife nutzen wollen. Beispiele dazu kennst du ja schon einige.

Wissen wir vorab nicht, wie viele Schleifendurchläufe nötig sind, brauchen wir eine **Schleife mit Bedingung**.

Beispiel: „Solange die x-Koordinate kleiner ist als die Breite des Fensters, wiederhole das Zeichnen des Quadrates und erhöhe den Wert der x-Koordinate um die Seitenlänge des Quadrates."

Achtung:

In der Zählschleife wurde die Zählvariable automatisch erhöht. Hier in der neuen Schleifenart müssen wir uns selbst um die Veränderung der Variable aus der Bedingung kümmern.

Beispiel:

```
while (x < width) {
  zeichneQuadrat(x,y,farbe);
  x=x+s; // der Wert der x-Koordinate wird um den Wert von s erhöht
}
```

In Worten haben wir diese Schleife oben schon beschrieben.

Würden wir die Erhöhung von x weglassen, wäre die Bedingung immer erfüllt und die Schleife würde zu einer sogenannten **Endlosschleife**. Das Programm stürzt dabei ab.

Auch while-Schleifen können ineinander verschachtelt werden.

```
while (y < height) {
  while (x < width) {
    zeichneQuadrat(x, y, farbe);
    x=x+s;
  }
  y=y+s;
}
```

„Solange die y-Koordinate kleiner ist als die Höhe des Fensters, **zeichne eine Reihe Quadrate bis zum Rand** und erhöhe dann y um die Seitenlänge des Quadrates."

Der fettgedruckte Teil wird durch die innere while-Schleife ausgeführt und entspricht dem Text von oben.

Weitere Aufgaben

Aufgabe 1:

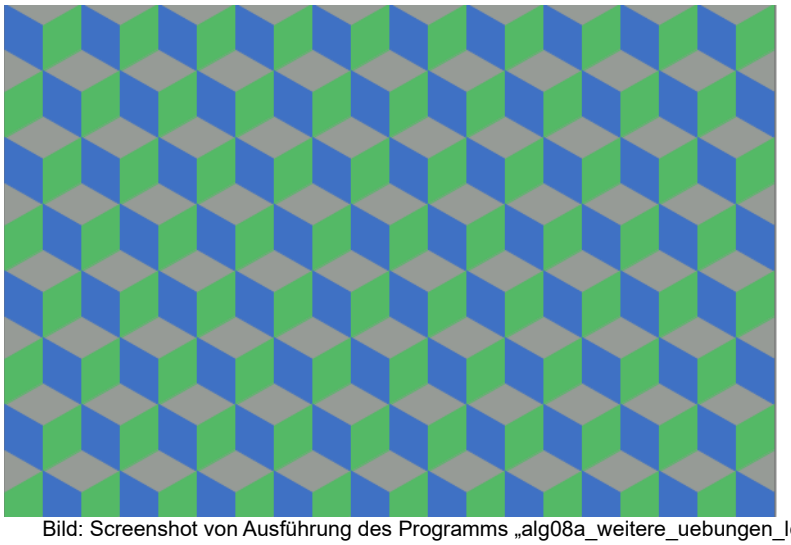


Bild: Screenshot von Ausführung des Programms „alg08a_weitere_uebungen_loesungen“ (Eisenmann)

1. *Überlege dir zunächst, aus welchen Figuren diese optische Täuschung besteht.*
2. *Schreibe eine Methode* `viereck()`*, der die Füllfarbe und die Koordinaten der vier Eckpunkte übergeben werden können. Nutze dazu die Methode* `quad()`[1]*.*
3. *Das Zeichnen der Rauten einer Farbe kann man sich sparen, wenn man zu Beginn den Hintergrund in dieser Farbe füllt. Überlege dir, welche Rauten dazu am geeignetsten sind und schreibe die* `setup()`*-Methode mit Einstellen der Fenstergröße und Färben des Hintergrunds (in der Vorlage: grau).*
4. *Programmiere dann das Zeichnen der Rauten der beiden anderen Farben (in der Vorlage: blau und grün im Wechsel).*
5. *Verändere ggf. so, dass das ganze Fenster gefüllt wird.*

1 Siehe Blatt „Methoden zum Zeichnen“

Aufgabe 2:

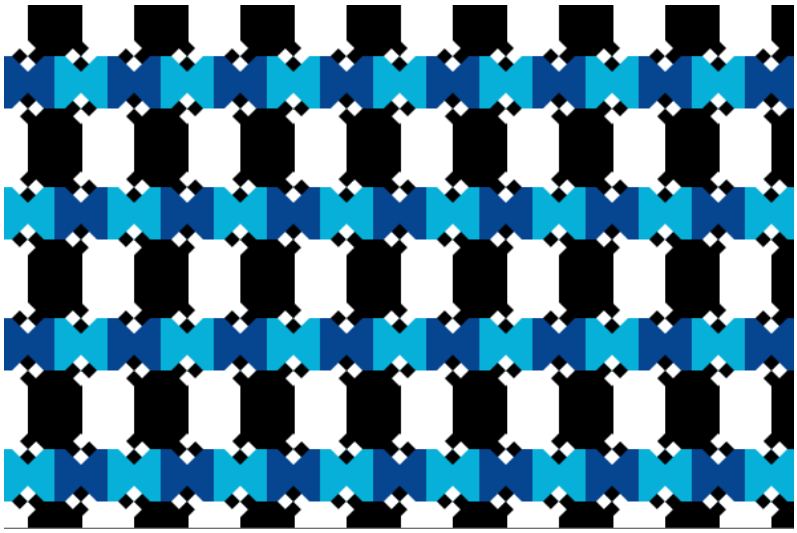


Bild: Screenshot von Ausführung des Programms „alg08b_weitere_uebungen_loesungen“ (Eisenmann)

1. *Programmiere zunächst die schwarzen, senkrechten Balken.*
2. *Programmiere die waagrechten Balken, in denen sich hell- und dunkelblaue Quadrate abwechseln.*
3. *Um die kleinen Vierecke – zusammengesetzt aus vier Rauten – zeichnen lassen zu können, brauchst du wie in Aufgabe 1 die Methode* `quad()`*. Schreibe eine Methode* `viereck()`*, der du die Farbe und alle Koordinaten der Eckpunkte übergeben kannst und dann eine Methode* `vier_rauten()`*, der eine Farbe und die Koordinaten eines Punktes übergeben werden können. Hier bietet sich der Punkt an, in dem sich alle vier Rauten treffen.*
   (Wenn du Aufgabe 1 gelöst hast, kannst du selbstverständlich die Methode `viereck()` *von dort nutzen.)*
4. *Ergänze dein Programm so, dass die optische Täuschung gezeichnet wird.*
5. *Schreibe ggf. eine zweite Version so, dass die optische Täuschung das ganze Fenster füllt.*

Aufgabe 3:

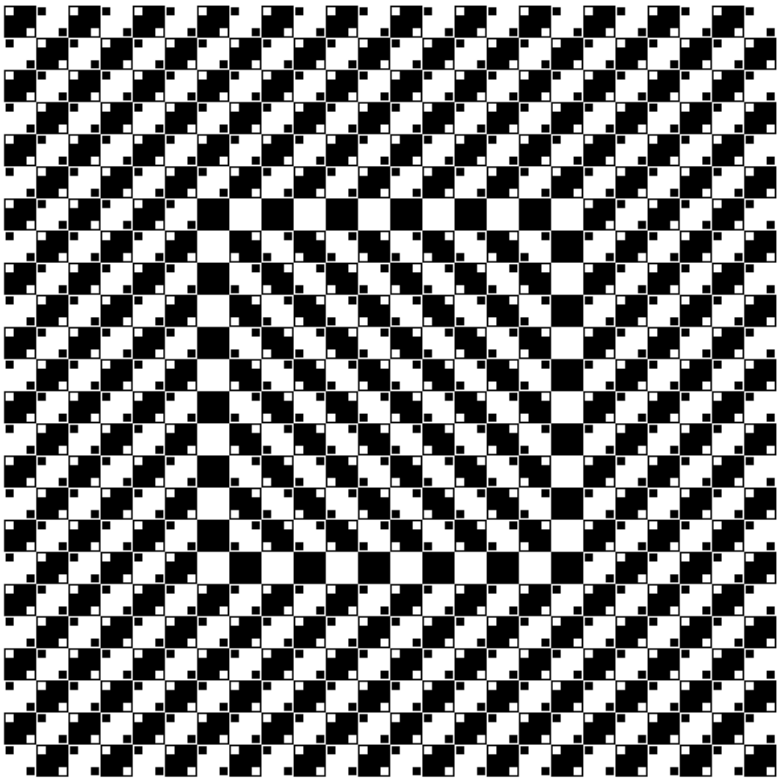


Bild: Screenshot von Ausführung des Programms „alg08c_weitere_uebungen_loesungen“ (Eisenmann)

1. *Schreibe eine Methode `zeichneQuadrat()`, der du neben der Startposition, die Grundfarbe und die Richtung der beiden kleinen Quadrate übergeben kannst.*
2. *Programmiere die optische Täuschung zuerst so, dass das ganze Feld so gefüllt wird, wie in den ersten Reihen.*
3. *Überlege dir anschließend, wie du die Mitte zeichnen lassen kannst und probiere deine Idee aus.*

Processing bietet noch viele weitere Möglichkeiten. :)

Aufgabe 4:

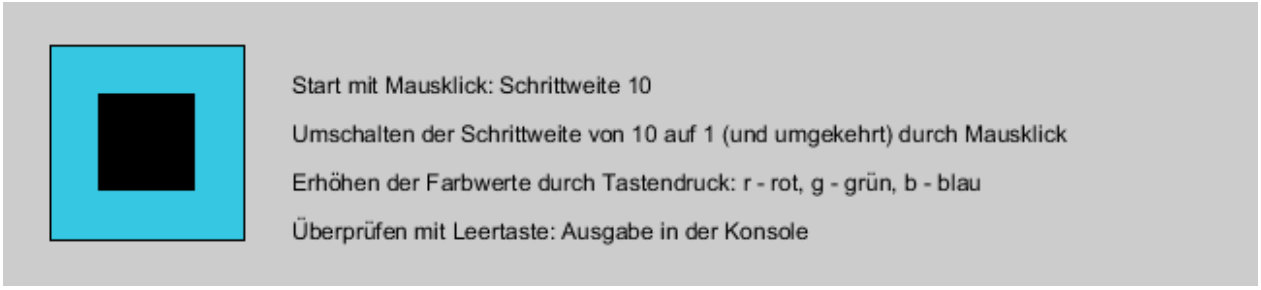


Bild: Screenshot von Ausführung des Programms „alg08d_weitere_uebungen_loesungen“ (Eisenmann)

Hier geht es um ein Farbenspiel. Das große Quadrat wird mit einer zufälligen Farbe gefärbt. Die einzelnen Farbwerte (rot, grün und blau) des inneren Quadrates kann man durch Tastaturdruck verändern (siehe Anleitung auf dem Screenshot). Mit der Leertaste (ASCII-Code: 32) wird überprüft und man bekommt die Information, ob die Farben zu viel, zu wenig oder genau richtig gewählt sind.

1. *Schreibe eine Methode `vorbereitung()`, in der du die beiden Quadrate zeichnen und den Text ausgeben lässt. Rufe diese Methode in der `setup()`-Methode auf. Nutze für die Farben jeweils zwei Variablen (z.B. `rot` und `rot_hintergrund`).*
2. *Für das Spiel brauchst du Maus- und Tastaturereignisse. Lies dir die Information unten auf diesem Blatt durch und programmiere dann Schritt für Schritt die nötigen Ereignismethoden. Denke dabei ans Testen! (Lasse dir z.B. die Farbwerte in der Konsole ausgeben: `println(…)`.)*

Möchte man in Processing **Maus- und Tastaturereignisse** nutzen, so braucht man die Methode `draw()`. Diese Methode wird ständig neu aufgerufen. Die Häufigkeit der Aufrufe gibt man in der `setup()`-Methode mit `frameRate(a)` vor. Der Wert a steht für die Anzahl der Aufrufe pro Sekunde.

Braucht man keinen ständigen Aufruf wie in unserem Programm hier in der Aufgabe, so nutzt man `noLoop()` in der `setup()`-Methode. Durch `redraw()` kann man dann aktiv die `draw()`-Methode aufrufen. In der Aufgabe bietet sich das bei jedem Tastendruck an.

Als Mausereignis brauchen wir `mousePressed()`, um zwischen den beiden Schrittweiten umzuschalten und als Tastaturereignis `keyPressed()`. Hier bekommt man bei jedem Tastendruck einen Wert in der Variable `key` übergeben. Zur Überprüfung der Buchstabentasten kann man direkt mit den Buchstaben arbeiten (z.B. `if (key == 'b')` …). Nutzt man Sondertasten, wie hier die Leertaste, so braucht man den zugehörigen ASCII-Code.

Die Mausposition bei den Mausereignissen erhält man mit `mouseX` und `mouseY`.

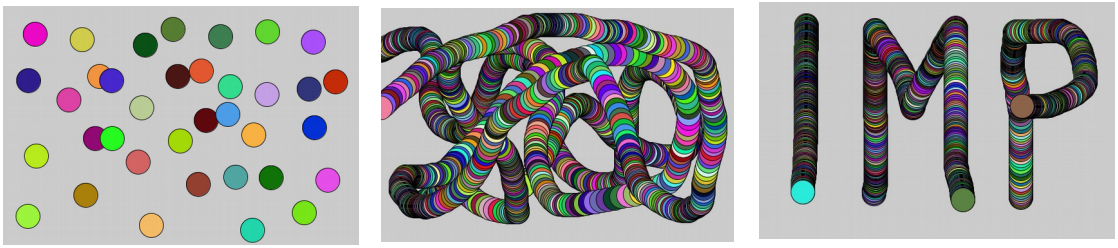
Weitere Informationen findest du in der Processing-Referenz[2] unter Input.

3. *Ergänze dein Programm nach Belieben (Versuche zählen; Möglichkeit, auch den Farbwert kleiner zu machen; …).*

2 *https://processing.org/reference/* (abgerufen am 26.04.2019)

Aufgabe 5:

Hier brauchst du die Informationen aus Aufgabe 4.



Bilder: Screenshots von Ausführung des Programms „alg08e_weitere_uebungen_loesungen“ (Eisenmann)

1. *Beim Klick auf das Fenster soll ein Kreis mit zufälliger Füllfarbe an der Mausposition gezeichnet werden. (Abbildung 1)*
2. *Wird die Maus über das Fenster bewegt (`mouseMoved()`), soll an der Mausposition ein Kreis mit zufälliger Füllfarbe gezeichnet werden. (Abbildung 2)*
3. *Nur bei gedrückter Maustaste wird gezeichnet (`mouseDragged()`). Gezeichnet wird dann wie in Aufgabe 2. (Abbildung 3)*
4. ****** Stelle Processing auf den Android-Modus um. Ändere die Größenangabe in der `setup()`-Methode auf `size(displayWidth,displayHeight)` und teste dann Nr. 3 auf dem Smartphone.*

Noch Lust auf mehr?

Aufgabe 1:

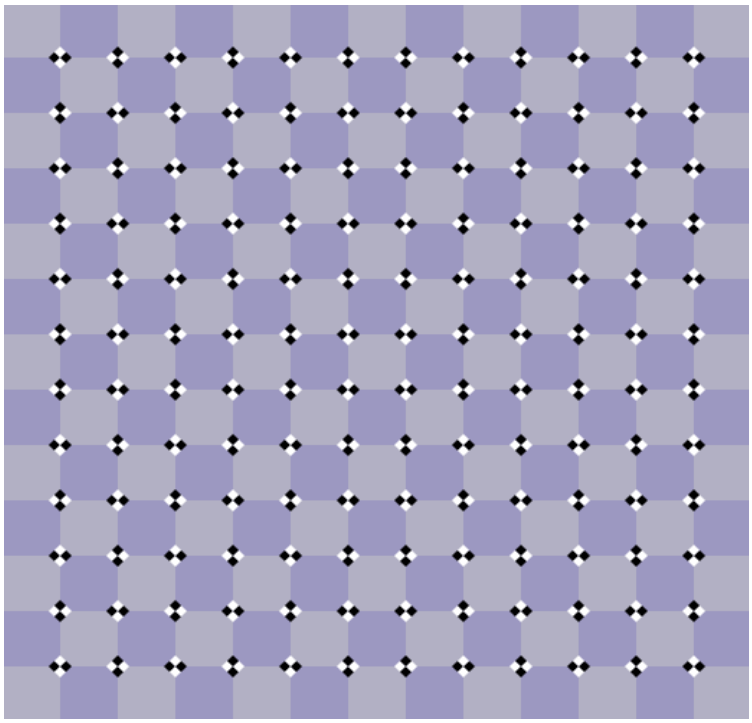
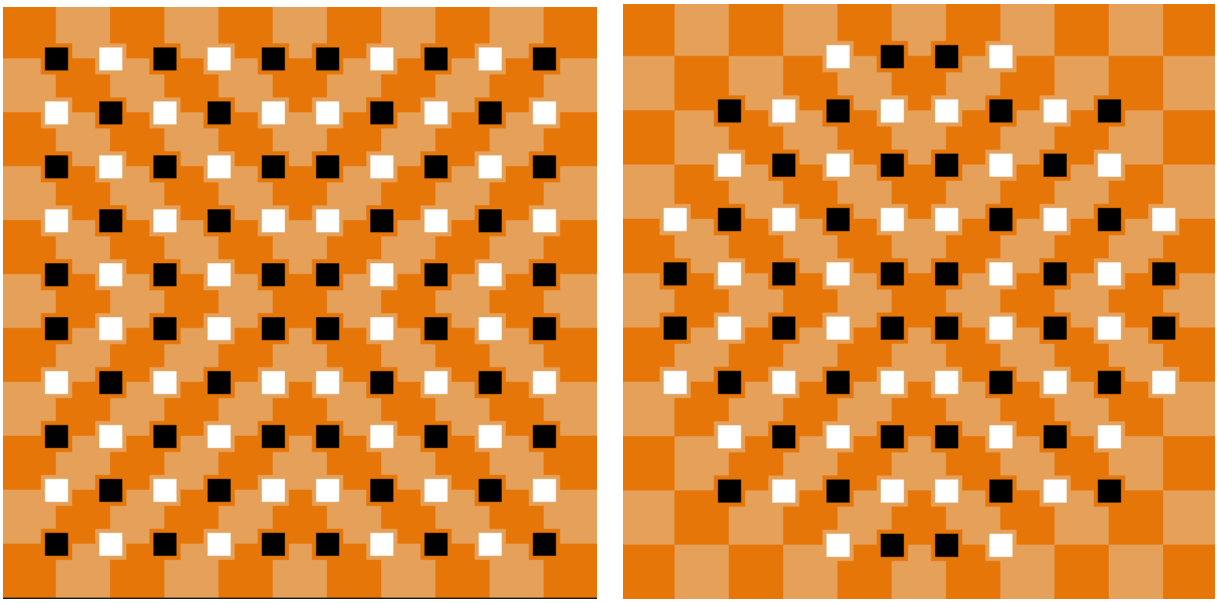


Bild: Screenshot von Ausführung des Programms „alg09a_loesung“ (Eisenmann)

Aufgabe 2:



Bilder: Screenshots von Ausführung der Programme „alg09b_loesung“ u. „alg09b_v2_loesung“ (Eisenmann)

Visualisierung von Arrays

Aufgabe: Ein Array von ganzen Zahlen (int) soll als Balkendiagramm dargestellt werden.

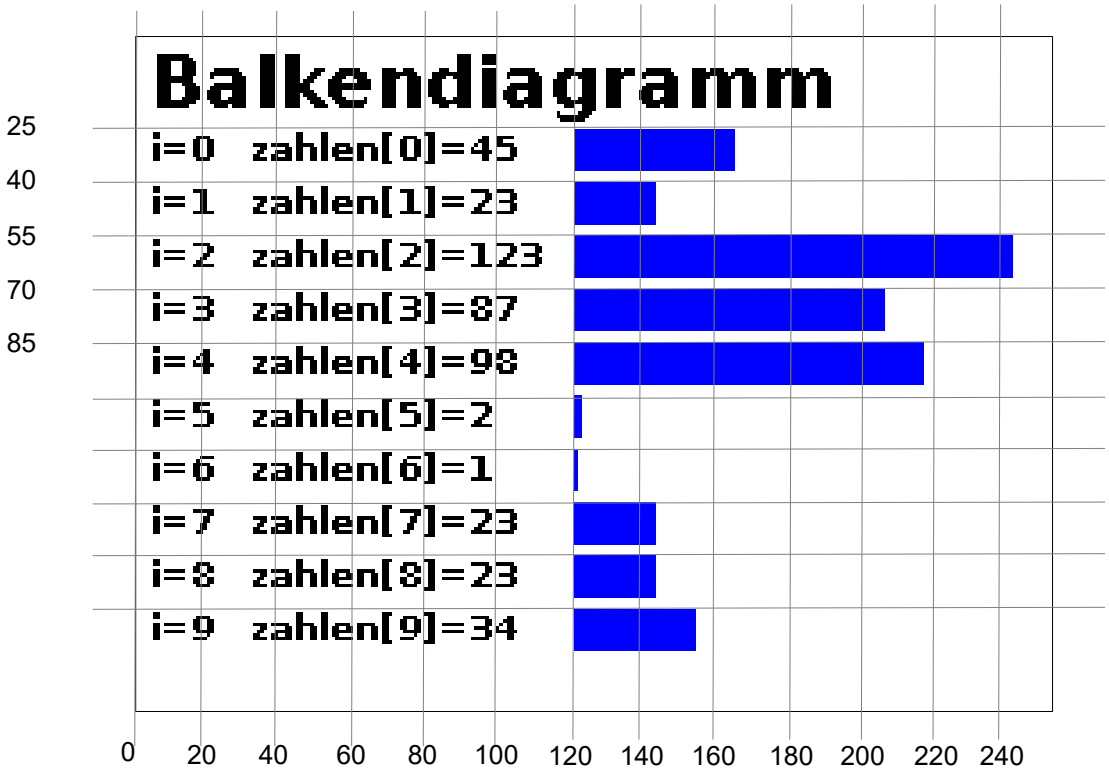


Bild: Screenshot von Ausführung des Programms „balkendiagramm“ (Schaller)

Du hast bei der "Kaffeehaus-Täuschung" die notwendigen Techniken für den Umgang mit Arrays schon angewendet. Schaue dir zunächst nochmals deine Lösung an.

Zunächst musst du die richtigen Koordinaten für die blauen Rechtecke bestimmen. Für den Befehl um ein Rechteck zu zeichnen, benötigst du die x- und die y-Koordinate des linken oberen Eckes und die Breite und Höhe des Rechtecks: `rect(x, y, width, height);`

	x-Koordinate	y-Koordinate	Breite	Höhe
i = 0	120	25		
i = 1				
i = 2				
i = 3				

Aufgabe:

1. *Lege in deinem Heft eine Tabelle wie oben an und fülle die leeren Tabellenfelder aus.*
   Damit man die Werte in einer Schleife verwenden kann, muss ein Term gefunden werden, der für jedes i als Ergebnis die Werte der Koordinaten, bzw. der Rechteckgröße liefert.



2. *Finde einen Term der Form m * i + c, bei der die obigen y-Koordinaten herauskommen.*
3. *Gib den Term an, mit dem man die Breite des Balkens unter Verwendung des zahlen-Arrays und i ausdrücken kann.*

Implementation:

Zur Erinnerung die ersten Schritte:

- *Jede Processing-Datei war unterteilt in Deklaration von Variablen, dem Schreiben der setup()-Methode und anschließend dem Schreiben eigener Methoden.*
- *Deklaration der Variablen: Wir brauchen ein Feld (Array), in dem ganze Zahlen abgelegt werden können. Im ersten Schritt sind diese vorgegeben.*

```
int[]   zahlen={45, 23, 123, 87, 98, 2, 1, 23, 23, 34};
```

- *Schreiben der setup()-Methode: Wir geben die Größe des Fensters und die Hintergrundfarbe vor und rufen dann unsere selbstgeschriebenen Methoden auf.*

```
void setup() {
  size(1000, 700);
  background(0);
  zeichneBalken();
}
```

- *Schreiben eigener Methoden: Unsere erste Methode soll die Balken zeichnen.*

```
public void zeichneBalken() {
}
```

4. *Version 1:*
   Implementiere die ersten vier Balken durch Angabe der Koordinaten aus der Tabelle. Ergänze dazu die Methode `zeicheBalken`*.*
5. *Version 2:*
   Ersetze die y-Koordinate und die Breite durch die oben ermittelte Formel. Deklariere vorher eine Integer-Variable `i`*. Setze i vor dem ersten Rechteck auf 0, vor dem zweiten auf 1 usw. Überprüfe, ob deine Rechtecke immer noch korrekt gezeichnet werden.*
6. *Version 3:*
   Verwende eine `for`*-Schleife, um alle Balken zeichnen zu lassen. Diese muss das* `i` *in jedem Durchgang um 1 erhöhen. Die Anzahl der Elemente im Array bekommst du durch* `zahlen.length`*.*

Nun soll noch der Text des Balkendiagramms ergänzt werden. Dazu gibt es einen Befehl

```
text(anzuzeigenderText, x, y);
```

also z.B.

```
text("Hallo", 10, 10);
text("x ist "+x, 20, 40);
```



Aufgabe:

7. *Bestimme wie oben die Koordinaten der einzelnen Texte. Überlege dir wieder einen Term, um die Koordinaten mit i auszudrücken.*
8. *Schreibe die Texte für i=0 bis i=3 ab. Markiere mit einem farbigen Stift die Teile, die in den Texten nicht identisch sind.*
   Die identischen Teile sind feste Texte. Sie werden in Anführungszeichen geschrieben. Die sich verändernden Teile sind Variablenwerte. Sie müssen an die festen Texte angehängt werden (siehe Abbildung rechts).
9. *Implementiere das Anzeigen des Textes.*

Das ist die **0**. Zeile.
Das ist die **1**. Zeile.
Das ist die **2**. Zeile.
=>

```
"Das ist die "+i+
". Zeile."
```

Am Ende jeder Implementation steht das Testen des Programms auf mögliche Fehler. Es reicht nicht, ein Beispiel erfolgreich auszuführen.

Aufgabe:

10. *Verändere die Einträge des Arrays.Verändere auch die Anzahl der Einträge.*
11. *Testen: Überlege dir, was passiert, wenn das Array gar keinen Eintrag hat. Was passiert, wenn ein Eintrag 0 ist? Was passiert bei negativen Einträgen? Teste diese Grenzfälle in deinem Programm. Trifft deine Erwartung zu?*
12. *Bewerte deine Tests: Können die Testfälle so überhaupt bei der Aufgabenstellung vorkommen? Ist eine Änderung des Programms notwendig?*

Bisher haben wir immer dieselbe Schriftart und Schriftgröße verwendet. Natürlich können wir auch das ändern.

Schritt 1: Deklaration einer neuen Schrift

```
PFont ueberschrift;
PFont beschriftung_balken;
```

Schritt 2: Erzeugen der Schrift mit Übergabe der Schriftart und -größe (z.B. in der setup()-Methode)

```
ueberschrift = createFont("Calibri", 48);
beschriftung_balken = createFont("Comic Sans", 28);
```

Schritt 3: Einstellen der Schrift vor der Textausgabe

```
textFont(ueberschrift);
text("Balkendiagramm", 2, 20);
```

Bemerkung: Die Schriftfarbe wird mit der bekannten Methode fill() geändert.

```
fill(0, 0, 255);
```

Wenn man wissen möchte, welche Schriftarten der Computer kennt, kann man das mit folgenden zwei Zeilen abfragen:

```
String[] fontList = PFont.list();
printArray(fontList);
```



Erzeugen von Zufallszahlen

Aufgabe: Ein Array soll mit Zufallszahlen gefüllt und dann angezeigt werden.

Umgang mit Arrays (Wiederholung):

1. Schritt: Deklaration

Um ein Array eines Variablentyps zu deklarieren, muss an den Typ eine eckige Klammer angehängt werden.

z.B.
```
double[] kommazahlen;
char[] buchstaben;
```

2. Schritt: Initialisierung

Es muss genügend Platz für alle Einträge im Speicher reserviert werden (Befehl `new`). Daher muss man jetzt die Größe des Arrays festlegen.

z.B.
```
kommazahlen = new double[20];
buchstaben = new char[100];
```

3. Schritt: Werte der Einträge festlegen.

In der Regel werden in einer `for`-Schleife alle Werte des Arrays einzeln belegt.

z.B.
```
for(int i=0; i < kommazahlen.length; i++) {
  kommazahlen[i] = 0.0;
}
```

Aufgabe Implementation:

Öffne dein Programm `balkendiagramm`. Speichere es unter einem neuen Namen `balkendiagramm_zufall` ab[1].

Ändere jetzt zunächst die Deklaration des Arrays `zahlen` ab, indem du die Initialisierung mit den vorgegebenen Zahlen löschst.

Das Feld wollen wir mit Zufallszahlen füllen. Wir müssen dazu eine neue Methode `erzeugeZufallsarray()` implementieren.

1. *Initialisiere das globale Array `zahlen` in der Methode `erzeugeZufallsarray()` so, dass es 4 ganze Zahlen speichern kann: `zahlen = new int[4];`*
2. *Fülle die vier Array-Elemente mit Zufallszahlen zwischen 1 und 300. Verwende dazu die Methode `random(int min, int max)`, die Zufallszahlen zwischen `min` und `max` erzeugt. Wir müssen diese noch in ganze Zahlen umwandeln: z.B. `zahlen[0] = int(random(1, 300));`*
3. *Rufe die Methode in der setup-Methode vor `zeichneBalken()` auf und teste sie.*
4. *Verwende jetzt statt der direkten Übergabe der Werte an alle Array-Elemente, eine Zählschleife, um das Feld mit den Zufallszahlen zu füllen.*

Um nicht auf die Länge eines Feldes festgelegt zu sein, schreiben wir jetzt unsere Methode so um, dass wir die Feldlänge übergeben können.

5. *Füge zunächst in der bisher noch leeren Klammer hinter dem Methodennamen `int laenge` ein.*
6. *Verwende den Parameter `laenge`, um die Größe des Arrays festzulegen. Ändere auch in der `for`-Schleife die obere Grenze entsprechend.*

1 Melde dich bei deiner Lehrerin / deinem Lehrer, falls du noch Fehler in deinem Programm hattest, damit du eine lauffähige Version bekommen kannst.



7. *Beim Aufruf der Methode musst du jetzt einen Wert mit übergeben. Teste mit mehreren Längen: 4, 10, 20,...*

Testen der Implementation:

8. *Überlege dir mögliche Grenzfälle. Bei welchen Werten von `laenge` könnte es Probleme geben? Bei welchen Zufallszahlen könnte es schwierig werden?*
9. *Vergleiche deine Überlegungen mit deiner Nebensitzerin / deinem Nebensitzer.*
10. *Testet eure Grenzfälle und passt ggf. das Programm daran an. Testet z.B. mit einer `if`-Anweisung, ob ein zulässiger Wert für `laenge` übergeben wurde.*



Externe Daten

Aufgabe: Ein Array soll mit Werten aus einer Datei gefüllt und dann angezeigt werden.

Externe Dateien: CSV oder XML?

1. *Schritt: Geeigneten Dateityp auswählen*
   CSV - Daten in Form einer Tabelle, in jeder Zeile hat man gleichartige Daten
   - Werte durch Komma(,) oder Strichpunkt (;) getrennt (comma separated values)
   - Texte z.T. in Anführungszeichen (")
   - können eine Zeile mit Überschrift enthalten
   XML - Daten in Form eines Baums (extended markup language)
   - geeignet, wenn nicht für jeden Eintrag die gleichen Daten vorliegen.
2. *Schritt: Datei laden/speichern*
   CSV - Verwende die Klassen Table.
   XML - Verwende die Klasse XML.
3. *Schritt: Übertrage die Werte in ein Array.*
   In der Regel werden in einer `for`-Schleife alle Werte des Arrays einzeln belegt.
   z.B.
```
for(int i=0; i < kommazahlen.length; i++) {
    // i => i. Zeile, 0 => 1. Spalte
   kommazahlen[i] = csv.getFloat(i,0);
}
```

Aufgabe externe Datei:

Lade die Datei punkte.csv mit einem Texteditor (nicht mit einer Tabellenkalkulation!):

1. *Untersuche, wie viele Zeilen und wie viele Spalten sie hat. Ist es eine csv-Datei mit Überschriften? Welches Trennzeichen wurde zwischen den Werten verwendet? Wurden alle Texte in Anführungszeichen eingeschlossen?*
   Ergänze die Datei um zwei eigene Einträge (so kannst du cheaten...).

Datentypen

Nachdem man die Struktur der Daten kennt, kann man sich überlegen, welche Datentypen dazu passen:

Art der Daten	Datentyp	Befehl zum Auslesen aus CSV-Datei
Ganze Zahlen	int	csv.getInt(i,"Punkte");
Dezimalzahlen	double oder float	csv.getFloat(i, "Messwert");
Texte	String	csv.getString(i, "Name");

Punktschreibweise

Die „Tabelle“ `csv` vom Typ `Table` hat bestimmte Fähigkeiten. Zum Beispiel kann man Werte aus einer bestimmten Zelle auslesen, wie oben in der rechten Spalte dargestellt. Dabei gibt i die Zeile an und z.B. „Punkte“ die Überschrift der gewünschten Spalte.

Zunächst müssen wir im Programm aber die csv-Datei laden:

```
Table csv = loadTable("punkte.csv","header, csv");
```



Dabei ist punkte.csv der Name der Datei, in der die Daten liegen. „header, csv“ gibt an, dass die Tabelle eine Überschrift besitzt (header) und als csv-Datei abgespeichert wurde.

Aufgabe Implementation:

*Lade deine **erste** fertige Datei `balkendiagramm` und speichere sie unter `balkendiagramm_csv` ab. Kopiere dann die Datei `punkte.csv` in den gleichen Ordner, in dem auch deine Processing-Datei liegt.*

2. *Lösche wieder beim Array `zahlen` die initialisierten Werte. Deklariere ein weiteres Array `namen` für die Spielernamen (welcher Typ ist geeignet?).*
3. *Schreibe eine Methode `ladeTabelle(String name).`*
4. *Lade zunächst die Daten aus der csv-Datei:*
   `Table csv = loadTable(name,"header, csv");`
5. *Initialisiere dann die Arrays in der richtigen Größe. Die Länge der Arrays muss der Anzahl der Zeilen mit Werten deiner csv-Tabelle entsprechen. Diese kannst du mit `csv.getRowCount()` abfragen.*
6. *Fülle sie dann mit den Werten aus der Tabelle in einer Zählschleife. Nutze dazu die Methoden `getInt(i,"Punkte")` und `getString(i,"Name")` für das Auslesen der Werte in der i-ten Zeile.*
7. *Ändere die Methode `zeichneBalken()` so, dass immer ein Name und daneben der Balken mit dem passenden Punktwert dargestellt werden.*
8. *Rufe vor der Methode `zeichneBalken()` in der setup-Methode die Methode `ladeTabelle(„punkt.csv“)` auf und teste dein Programm.*

Testen der Implementation:

Da hier Daten aus einer externen Datei kommen, können hier viele Fehler passieren. Zum Beispiel könnte die Datei gar nicht existieren oder keine Überschriften enthalten.

9. *Überlege dir noch zwei Fehler, die die Daten haben könnten.*
10. *Kopiere die Datei punkte.csv, passe das Programm so an, dass deine Kopie geladen wird, und experimentiere, was bei den Fehlern passiert.*
11. *Lies die Dokumentation zum Befehl, der zum Laden der Datei verwendet wird, und versuche das Programm so anzupassen, dass es mit einer Datei ohne Überschriften zurecht kommt. (https://processing.org/reference/loadTable_.html)*
12. *Versuche möglichst viele Fehler abzufangen (z.B. das Fehlen der Datei).*



Smarties auszählen...

Aufgaben:

1. *Stellt die importierten Daten (Farbe, Anzahl) in einem Balkendiagramm dar.*

Die folgenden Aufgaben können in beliebiger Reihenfolge bearbeitet werden. Aufgaben mit Sternchen sind freiwillig.

2. ***** Nutzt ein weiteres Feld (Array) für die Farben (z.B. im Hexadezimalcode) und färbt Schrift und Balken passend.*
3. *Überlegt euch, wie man das Maximum bestimmen könnte. (In Klasse 8 habt ihr das beim Froschhüpfen schon mal gemacht…) Schreibt dann eine Methode sucheMaximum(), die nach der größten Anzahl sucht und diese ausgibt (z.B. in der Konsole unten mit println(…), oder ihr umrandet den Balken, lasst ihn blinken o.ä.).*
4. *Berechnet für alle Farben die relative Häufigkeit und lasst diese mit ausgeben. (Achtung: Für Kommazahlen braucht ihr den Datentyp double.)*
5. ***** Stellt die Verteilung der Farben in einem einzigen Stabdiagramm dar. (Wenn z.B. 10% der Smarties rot sind, färbt ihr 10% des Stabes rot, etc. Das Stabdiagramm könnt ihr durch aneinandergezeichnete Rechtecke darstellen.)*
6. *Nutzt Aufgabe 3, um eure Anzahlen zu sortieren. Schreibt dazu eine Methode sortiereArray(), die in ein weiteres Zahlenfeld die sortierten Anzahlen einträgt, und lasst die Daten dann sortiert anzeigen.*

Ihr könnt gerne noch weitere Ideen verwirklichen versuchen.